TECHNICAL DOCUMENTATION

AI Interview Simulator Platform

Complete file-by-file reference and functional specification

Repository d:\AI INTERVIEW · branch `main`

Backend Node.js · Express 5 · TypeScript · Prisma 7 · PostgreSQL · Socket.IO · Playwright

Frontend Next.js 14 App Router · React 18 · TypeScript · Tailwind CSS · Recharts · Monaco

AI services Groq & Mistral (LLM) · Deepgram (STT + TTS) · Microsoft Edge Neural (TTS)

Scale ~30,700 lines of application source across ~120 files

Generated 20 August 2026 · Describes every source file, what it does, and the reasoning behind its design.

Contents

1 · The system in one page

> An AI interviewer that runs complete first-round interviews on its own. A recruiter describes the role once; the platform writes the question set, schedules the interviews, conducts them as a spoken conversation, evaluates the candidate across technical, communication and behavioural dimensions, and produces a scored report with a hiring recommendation.

1.1 What it does

Capability	How it is delivered
Session creation	Recruiter enters job description, skills, seniority, round type, schedule, duration, interviewer personality, language, pass mark. A real Google Meet / Zoom / Teams meeting is created through that provider's API, or an existing link is pasted.
Question generation	An LLM writes an intro / behavioural / technical / scenario / project / coding set sized to the duration and calibrated to the seniority. A deterministic template set is the fallback when the model is unavailable.
Candidate intake	One at a time, or bulk CSV/XLSX import with per-row overrides, column-alias tolerance and an editable preview of every problem found.
Resumes	PDF / DOCX / TXT is extracted, stored in Postgres, analysed against the job description into a structured profile, and turned into per-candidate questions spliced into the project round.
The interview	A spoken conversation. A state machine owns the script; an interviewer service owns the reactions. Follow-ups, rephrases, clarifications, doubt handling, identity verification and a real closing Q&A.
Coding	A generated problem, a Monaco editor, real execution of JavaScript / Python / C++ / Java against visible and hidden test cases in a sandboxed child process, then an LLM code review.
Signals	Rule-based speech analysis (pauses, fillers, hedges, pace, lexical variety, ASR confidence), real-time insight events, browser-side video presence/gaze/motion metrics, and tab-switch proctoring.
Reporting	Weighted overall score with hard gates, evidence-cited strengths and weaknesses, PDF and Excel export, candidate feedback email, ATS webhook push, ranking, shortlist and side-by-side comparison.
Automation	Invitations, 24 h / 1 h / 5 min reminders, no-show nudges, absent marking, session activation and closure, and a self-healing evaluation retry queue — all driven by a one-minute scheduler tick.

1.2 Two ways an interview runs

The same interview engine drives both paths. Only the transport differs.

	Built-in browser room	Meeting bot
Where	The platform's own page at `/interview/<token>`	Inside a Google Meet, Zoom or Teams call the recruiter already created
Who speaks	The candidate's browser, via the Web Speech synthesis API	A Playwright-driven Chromium that joins as a participant, with server-side TTS injected as a synthetic microphone
Who listens	Microphone → MediaRecorder → Socket.IO → server → Deepgram; browser recognition as fallback	Remote WebRTC tracks tapped in-page → PCM → server → Deepgram
Coding round	Rendered inline in the room	A separate browser tab; the link is posted into the meeting chat, and a read-only mirror can be screen-shared back into the call
Owner	`realtime/interviewGateway.ts`	`services/meetingBot/MeetBotSession.ts`
Shared	`InterviewStateMachine`, `LiveInterviewerService`, `TranscriptService`, `InsightService`, `EvaluationQueue`, `EvaluationService`	

WHY THE SPLIT MATTERS

The state machine decides *what* comes next and cannot be talked out of it, while the interviewer service decides only *how to react* to an answer. The model therefore cannot skip questions, reveal the question bank, or end the interview early — those are structural guarantees enforced in code, not prompt requests.

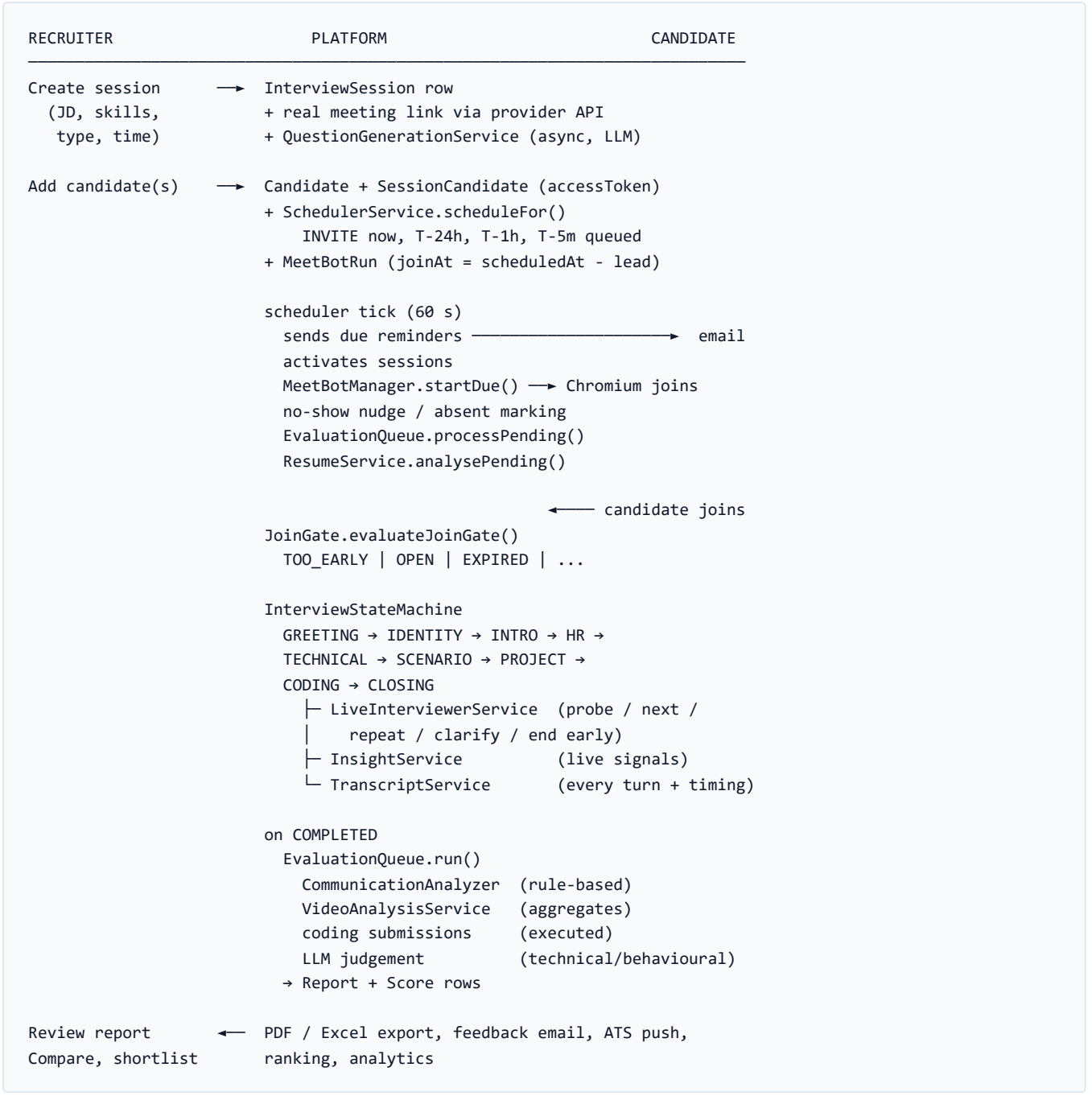
1.3 Repository layout

```
d:\AI INTERVIEW
├── backend/                        Express 5 + Prisma + PostgreSQL + Socket.IO
│   ├── prisma/
│   │   ├── schema.prisma           568 lines — 17 models, 13 enums
│   │   └── seed.ts                 Demo recruiter, sessions, candidates
│   ├── scripts/                    18 verification / operations scripts
│   ├── src/
│   │   ├── server.ts               Process entry: HTTP, Socket.IO, boot checks
│   │   ├── routes/index.ts         Every REST route, in one file
│   │   ├── controllers/            8 controllers — the HTTP surface
│   │   ├── services/               25 domain services
│   │   │   └── meetingBot/         The Playwright interviewer (16 files)
│   │   ├── realtime/               3 Socket.IO namespaces
│   │   └── lib/                    env, prisma, auth, http, ai, email, providers
│   ├── Dockerfile                  Playwright base image + g++ + JDK + x11vnc
│   └── fly.toml                    Fly.io machine, volume and health config
│
├── frontend-v2/                    Next.js 14 App Router (the only frontend)
│   └── src/
│       ├── app/(app)/              Authenticated recruiter pages
│       ├── app/(auth)/             Login / register
│       ├── app/interview/[token]/  Candidate room + coding editor + mirror
│       ├── components/             UI kit, charts, session tabs, interview UI
│       ├── hooks/                  speech, audio streaming, video, proctoring, bot
│       └── lib/                    api client, formatting, meet-interview types
│
├── docs/                           Deployment (Render / Fly / Cloud Run / Koyeb),
│                                   environment reference, meeting-bot guide
├── render.yaml                     Render blueprint for the backend
├── .github/workflows/              Fly.io deploy on push to main
└── package.json                    Root scripts that drive both apps
```

NOT PART OF THE APP

`temp-repo/` is an unrelated scratch checkout. `backend/.meet-bot-profile/` is the bot's signed-in Chromium user-data directory — generated data, not source. `backend/uploads/` holds bot-failure screenshots and a probe file.

1.4 End-to-end lifecycle



2 · Data model

backend/prisma/schema.prisma — 568 lines · PostgreSQL · 17 models · 13 enums

2.1 Entity map

```
User (recruiter)
 └── InterviewSession                    the role + schedule + behaviour config
      ├── QuestionSet ── Question        the shared, JD-driven question list
      └── SessionCandidate  ◄── Candidate  THE INTERVIEW ITSELF
           │                             (token, status, resume, retry state)
           ├── InterviewTranscript ── TranscriptTurn
           ├── RealtimeInsight             live behavioural signals
           ├── VideoAnalysisFrame          sampled webcam aggregates
           ├── CodingSubmission ── Question  executed code + review
           ├── Reminder                    invite / T-24h / T-1h / T-5m / nudge
           ├── MeetBotRun                  one bot launch, locked per process
           └── Report ── Score             the scored outcome

User
 └── AtsIntegration ── AtsSyncLog          outbound webhook config and history
```

THE CENTRAL ROW

`SessionCandidate` is the join between a candidate and a session, and it *is* the interview: it owns the access token, the transcript, the report, the submissions, the resume and the retry state. Almost every query in the system is scoped through it, and ownership checks are always written as `interviewSession: { userId }` rather than trusting an id in the URL.

2.2 Every model and enum

Accounts

Model / enum	Fields and purpose
`UserRole`	`RECRUITER` · `MENTOR` · `ADMIN`
`User`	`id, email (unique), password (bcrypt), name, company, role, createdAt, updatedAt`. Owns interview sessions and ATS integrations; both cascade on delete.

Sessions

Model / enum	Fields and purpose
`InterviewType`	`TECHNICAL` · `HR` · `MIXED` — drives the question mix and the scoring weights.
`SessionStatus`	`DRAFT` · `SCHEDULED` · `ACTIVE` · `COMPLETED` · `CANCELLED`
`InterviewerPersonality`	`FRIENDLY` · `NEUTRAL` · `FORMAL` · `CHALLENGING` — selects the persona and its probing bias.

Model / enum	Fields and purpose
`InterviewSession`	`title, jobDescription, skills[], experienceLevel, type, scheduledAt, durationMinutes, status` ; meeting fields `meetingProvider, meetingLink, externalEventId` ; behaviour fields `personality, language, codingEnabled, videoAnalysisEnabled, passMark` . Indexed on `(userId, scheduledAt)` and `status` . `meetingProvider` is a plain string, not an enum, because rows written before the built-in room was removed still say `BUILT_IN` .

Candidates and the interview row

Model / enum	Fields and purpose
`Candidate`	`name, email (unique), mobile` . Shared across sessions, so one person interviewed twice keeps one record and one history.
`CandidateStatus`	`INVITED · JOINED · IN_PROGRESS · COMPLETED · INCOMPLETE · ABSENT · CANCELLED` . Only `COMPLETED` produces a score; `INCOMPLETE` means the candidate left before the interviewer finished and is deliberately never scored.
`SessionCandidate`	**Identity** `accessToken` (unguessable UUID in the candidate URL), `identityVerified` . **Timeline** `invitedAt, joinedAt, startedAt, completedAt, absentAt` . **Evaluation retries** `evaluationAttempts, evaluationError, evaluationRetryAt` . **Resume** `resumeFileName, resumeFile (Bytes), resumeMimeType, resumeSizeBytes, resumeText, resumeProfile (Json), resumeQuestions (Json), resumeParsedAt` . Unique on `(interviewSessionId, candidateId)` .

> **WHY RESUMES LIVE IN POSTGRES**
>
> They are small and they are part of the hiring record. Keeping the bytes with the interview they belong to means the file cannot disappear independently of the decision it informed, and there is no bucket to configure, expire or lose.

Questions

Model / enum	Fields and purpose
`QuestionCategory`	`INTRO · IDENTITY · HR · TECHNICAL · SCENARIO · PROJECT · CODING · CLOSING`
`QuestionDifficulty`	`EASY · MEDIUM · HARD`
`QuestionSet`	One per session (`@unique`), with `generatedBy` recording the model that produced it, for auditability.
`Question`	`content, order, category, difficulty, skill, expectedAnswer, meta (Json)` . `meta` is the category-specific payload: coding test cases, constraints, starter code, optimal complexity; behavioural expected points and red flags.

Transcript and live signals

Model / enum	Fields and purpose
`Speaker`	`AI · CANDIDATE · SYSTEM`
`InterviewTranscript`	One per interview, plus an LLM-written `summary` that makes the recruiter list scannable.

Model / enum	Fields and purpose
`TranscriptTurn`	`speaker, text, questionId, round, latencyMs, durationMs, confidence, wordsPerMinute, timestamp`. The timing columns are what makes communication scoring measurable rather than an opinion.
`InsightType`	`HESITATION · LONG_PAUSE · UNCLEAR_RESPONSE · LOW_CONFIDENCE · HIGH_CONFIDENCE · FILLER_HEAVY · OFF_TOPIC · STRONG_ANSWER`
`RealtimeInsight`	`type, message, severity (0..1), meta` — emitted per answer and aggregated into a composure score.
`VideoAnalysisFrame`	`facePresence, motion, gazeStability, expression, confidence, capturedAt` — aggregates only. No imagery is ever transmitted or stored.

Coding, evaluation and reporting

Model / enum	Fields and purpose
`CodingSubmission`	`language, code, passedCases, totalCases, runtimeMs, executionResult (Json), timeComplexity, spaceComplexity, qualityScore, correctnessScore, reviewFeedback`. Hidden-case results are recorded here for the recruiter but never returned to the candidate.
`HiringRecommendation`	`STRONG_HIRE · HIRE · CONSIDER · REJECT`
`Report`	`overallRating, technicalScore, communicationScore, behavioralScore, codingScore?, videoConfidenceScore?, hiringRecommendation, recommendationReason, aiFeedback` (for the recruiter), `candidateFeedback` (written to the candidate), `summary`, `details (Json)`, `feedbackEmailSentAt`.
`Score`	Flat rows of `category, label, value, maxValue, evidence` — *Overall*, *Summary*, *Communication*, *Technical*, *Behavioral* and one *Skill* row per skill actually discussed. Powers the analytics skill breakdown without re-parsing the JSON blob.

Scheduling

Model / enum	Fields and purpose
`ReminderKind`	`INVITE · T_MINUS_24H · T_MINUS_1H · T_MINUS_5M · NO_SHOW_NUDGE · FEEDBACK`
`ReminderStatus`	`PENDING · SENT · FAILED · SKIPPED`
`Reminder`	Unique on `(sessionCandidateId, kind)` — that constraint is what guarantees a nudge fires exactly once per candidate, no matter how many scheduler ticks overlap.

The meeting bot

Model / enum	Fields and purpose
`MeetBotStatus`	`SCHEDULED · STARTING · OPENING_MEETING · PRE_JOIN · WAITING_FOR_ADMISSION · JOINED · WAITING_FOR_CANDIDATE · INTRODUCTION · QUESTIONING · FOLLOW_UP · FINAL_QUESTION · ENDING · COMPLETED · FAILED · CANCELLED`

Model / enum	Fields and purpose
`MeetBotRun`	`meetLink` (held per run, so replacing a link never rewrites the history of a run that already happened), `platform, status, statusDetail, errorCode, errorMessage, joinAt, startedAt, joinedAt, candidateSeenAt, endedAt, attempts`, and the distributed lock `lockedBy / lockedAt`.

> **WHY THE LOCK COLUMNS EXIST**
>
> Two bots joining one meeting would talk over each other and write two transcripts to the same interview. `lockedBy` is claimed with a single conditional `updateMany` — condition and write in one statement — so two replicas, or two overlapping scheduler ticks, cannot both see the run as free.

ATS

Model / enum	Fields and purpose
`AtsProvider`	`GREENHOUSE · LEVER · WORKABLE · WEBHOOK`
`AtsIntegration`	`provider, name, webhookUrl, apiKey, enabled`. The stored key is never returned by the API — only a `hasApiKey` boolean.
`AtsSyncLog`	`reportId, success, responseStatus, message, createdAt` — the last 50 are shown per integration.

3 · Backend — process & infrastructure

3.1 Process entry

backend/src/server.ts 157 lines

Builds the Express app, mounts the API, attaches Socket.IO to the same HTTP server, and performs a set of boot-time checks whose whole purpose is to surface misconfiguration before it costs an interview.

What it wires

- **Middleware** — `cors({ origin: true, credentials: true })` (any origin may serve the candidate room, so the request origin is reflected rather than pinned), `express.json({ limit: '2mb' })`, urlencoded, `cookie-parser`.
- **Health** — `GET /health` runs `SELECT 1` and returns database liveness, `emailSender` status, `speechToText` status, the live LLM provider and both model names, active interview count, meeting-bot enablement / voice / running / capacity / unsupported-host, uptime and timestamp. Returns 503 when the database is unreachable.
- **API** — `app.use('/api', apiRoutes)`, then `notFoundHandler`, then `errorHandler`.
- **Socket.IO** — one server with `pingTimeout: 60 s` and `pingInterval: 25 s`, because long interviews sit idle while the candidate thinks. Three namespaces are configured: `/interview`, `/meet-bot`, `/coding`.

Boot sequence after `listen()`

1. `verifyEmailSender()` — asks Brevo for the account's verified senders.
2. `verifySpeech()` — confirms Deepgram accepts the key.
3. `warnAboutHost()` — if the meeting bot is enabled on a serverless host, prints a multi-line explanation of why it cannot work there.
4. `warnAboutAppUrl()` — if the API is local but `APP_URL` is remote, warns that candidate links (especially the coding editor) point at a deployment that may not have the page.
5. `MeetBotManager.recoverOrphans()` → `SchedulerService.start()` → `logUpcoming()` → `warmBrowserBinary()`, in that order.

> **WHY THAT ORDER**
>
> Interviews whose process died still hold a lock and an in-progress status; clearing them *before* the scheduler starts lets them be retried instead of sitting "running" forever with no browser behind them. The queue is printed after the scheduler is armed so the list shown is the list it will actually act on. Chromium is then pulled off disk while nothing is waiting — a measured 40-second first launch versus 0.4 s afterwards.

Shutdown

`SIGINT` / `SIGTERM` stop the scheduler, ask every running bot to leave its meeting and close its browser, close Socket.IO and the HTTP server, and disconnect Prisma. Without the bot shutdown, Chromium processes outlive the server and keep their profile directories locked. An `unhandledRejection` handler logs rather than crashes.

3.2 Route table

backend/src/routes/index.ts 152 lines

Every REST route in one file, deliberately, so the surface is readable at a glance. Two `multer` instances are configured: `sheetUpload` (memory, 5 MB) for CSV/XLSX and `resumeUpload` (memory, 10 MB) with a filter accepting PDF, DOCX, DOC, TXT and MD by MIME type *or* extension.

The file is split by authentication:

- **Public** — auth endpoints, and the candidate-facing `/interview/:token/*` family (context, transcript, coding challenge, run code, hint, video metrics, resume status, resume upload). These authenticate by the opaque access token in the URL.
- `router.use(requireAuth)` — everything below needs a recruiter JWT: sessions, questions, candidates, ranking and export, meeting interviews, live inspection and evaluation, reports, analytics and ATS.

> **ORDERING DETAIL**
>
> `/interviews/queue`, `/interviews/import/template`, `/interviews/import/preview` and `/interviews/bulk` are declared *before* `/interviews/:id`, or the literal paths would be swallowed by the parameterised route.

3.3 The `lib/` layer

backend/src/lib/env.ts 234 lines

Loads `backend/.env` and validates *every* variable with a Zod schema. On failure it prints the flattened field errors and calls `process.exit(1)` — the process refuses to start on an invalid configuration rather than failing hours later, mid-interview.

A `superRefine` enforces the LLM key rule: naming a provider without its key fails at boot, and in `auto` mode at least one of `GROQ_API_KEY` / `MISTRAL_API_KEY` must be present. The file also exports `isMeetingProviderConfigured`, a three-boolean summary of whether Google, Zoom and Teams have complete credential sets.

Every meeting-bot timing constant is documented with the measurement that produced it — notably `MEET_BOT_WARMUP_SECONDS = 75`, derived from real runs where launching Chromium and loading the meeting page took between 9 and 62 seconds.

backend/src/lib/prisma.ts 63 lines

A single `PrismaClient` over a `pg` `Pool`, cached on `globalThis` in development so hot reload does not exhaust connections.

> **WHY THE POOL IS TUNED THIS WAY**
>
> Serverless Postgres (Neon and friends) drops idle connections on its own side. Without `idleTimeoutMillis: 10 s` (well under the provider's cutoff), `connectionTimeoutMillis`, `query_timeout`, `statement_timeout` and `keepAlive`, `pg` keeps handing out sockets the far end has already closed and queries hang forever instead of failing — which in this app meant a candidate sitting in a silent interview room. A `pool.on('error')` listener is mandatory: without it an idle-client error is an unhandled `'error'` event and Node tears the process down.

Exports `checkDatabase()`, the cheap liveness probe used by `/health`.

backend/src/lib/auth.ts 52 lines

JWT helpers and middleware. `signAccessToken` issues a 30-minute token; `signRefreshToken` a 7-day one. `requireAuth` rejects a missing or invalid `Bearer` token with 401; `optionalAuth` reads the user when present and never rejects.

backend/src/lib/http.ts 87 lines

The error vocabulary: an `HttpError` class plus `badRequest`, `unauthorized`, `forbidden`, `notFound`, `conflict`. `param(req, name)` collapses Express 5's `string | string[]` parameter type. `asyncHandler` wraps async routes so rejected promises reach the error middleware instead of hanging the request.

`errorHandler` maps failures to honest status codes:

Condition	Response
Error carries `isRateLimit`	**429** with a `Retry-After` header and a message naming the wait in minutes, plus `code: 'LLM_RATE_LIMITED'`
`ZodError`	**400** with per-field `{ path, message }` details
`HttpError`	Its own status and message
Prisma `P2002` / `P2025`	**409** / **404** — a unique-constraint violation is a client error, not a server fault
anything else	**500**, logged in full server-side, opaque to the caller

backend/src/lib/ai.ts 323 lines

The single door to every language-model call. Both Groq and Mistral speak the OpenAI chat-completions shape, so one small `fetch` serves both.

> **WHY THE VENDOR SDK WAS DROPPED**
>
> It hard-codes `/openai/v1/chat/completions` — Groq's path and nobody else's — so pointing it at another host quietly produces 404s.

Provider routing

`resolveRouting()` reads `LLM_PROVIDER`. Pinned to one provider, both roles go there. In `auto` with both keys present the roles split by what each is good at: **Groq answers the fast, conversational turns** (its hardware is several times quicker and the candidate is waiting on every one) while **Mistral does the heavy off-the-clock work** — question generation, evaluation, script translation — where its roomier quota matters more than speed. With one key, everything runs there. `/health` reports the resulting label.

Rate-limit handling

- `RateLimitError` parses the provider's own hint — `"try again in 1h5m25.152s"` — into `retryAfterSeconds`, handling hours, minutes and seconds independently.
- `llmCooldown` keeps a **per-provider** cooldown map. A quota block on Groq's conversation budget must not pause Mistral's evaluations, and vice versa. A cooldown is never shortened, only extended.
- `assertNotCoolingDown()` throws immediately if the target provider is known to be blocked, so callers stop rediscovering the block — which otherwise floods the log and burns a request per retry.

Entry points

Export	Behaviour
`complete()`	One chat completion, returning trimmed text. 90-second abort timeout.
`completeJson()`	Requests `response_format: json_object`, parses and validates against a Zod schema. On a parse or validation failure the *error itself* is fed back to the model as a repair instruction and the call retries (default 2 retries). A rate limit is **not** retried — retrying immediately would burn the remaining attempts and bury the real cause.
`extractJson()`	Strips markdown fences and surrounding prose, returning the outermost JSON object.
`unquote()`	Drops quotation marks wrapping a whole reply. Asked for a line to say, these models often answer `"Excellent answer, thank you."` — quotes included — and those quotes end up in the transcript and in the mouth of the interviewer. Only stripped when they enclose the entire string.
`schemaHint()`	Renders a compact JSON-shape hint to append to a prompt.

3.4 Email and meeting providers

backend/src/lib/email/EmailService.ts 264 lines

Transactional email through Brevo, with a shared HTML shell and button helper. Four typed messages: `sendInvite` (time, duration, join button, optional coding-editor link), `sendReminder` (24 h / 1 h / 5 min), `sendNoShowNudge`, and `sendFeedback` (overall rating, strengths, improvements, closing message). Without `API_KEY_FOR_EMAIL` every message is printed to the log instead, so local development still exercises the full scheduling path.

THE ONE SETTING THAT SILENTLY BREAKS

Brevo answers `201 Created` and *then* discards the message if `EMAIL_FROM` is not a verified sender. Nothing in the response says so. `verifyEmailSender()` therefore lists the account's active senders at boot and caches them; `send()` refuses up front rather than attempting a send it knows will be rejected, turning a silent loss into an error the scheduler can retry and log. `/health` reports `verified`, `unverified`, `unchecked` or `disabled`.

backend/src/lib/providers/
MeetingProvider.ts · GoogleMeetProvider.ts · ZoomProvider.ts · TeamsProvider.ts · index.ts

Creates a *new* meeting through each vendor's API — distinct from `services/meetingBot/`, which *joins* a meeting that already exists.

Provider	Mechanism
Google Meet	Calendar v3 `events.insert` with `conferenceDataVersion: 1` and a `hangoutsMeet` create request, authenticated by an OAuth2 refresh token. Returns `hangoutLink`.
Zoom	Server-to-server OAuth (`account_credentials`, Basic client auth), then `POST /v2/users/me/meetings` with `type: 2`, `join_before_host: true`, no waiting room. Tokens are short-lived so one is fetched per call.
Teams	Client-credentials token from Microsoft Entra, then Graph `POST /users/{organizer}/onlineMeetings`. Returns `joinWebUrl`.

WHY THERE IS NO FALLBACK ANY MORE

`createMeeting()` used to substitute the platform's own browser room whenever a provider was unconfigured or its API said no. The reasoning was wrong in one specific way: the fallback was *silent*. A recruiter who asked for Zoom got a scheduled interview, an invitation email, and no indication that the link inside it was not a Zoom link at all. It now throws `MeetingCreationError`, which the session controller turns into a 400 the recruiter can act on.

backend/src/lib/candidateFields.ts 45 lines

Field rules shared by the single-candidate form and the spreadsheet import, so a row typed by hand and the same row in a CSV are judged identically. `mobileField` counts digits (7–15) and allows `+ - ( ) . /` rather than matching a shape — the number is never dialled by the platform, so the cost of rejecting a valid-but-unusual format is real and the benefit of strictness is nil. `emailField` trims and lowercases, because an email is a key here and casing is not.

4 · Backend — HTTP controllers

> Eight controllers, ~2,400 lines. Each one validates with Zod, scopes every query through the owning user, and delegates the real work to a service.

backend/src/controllers/auth.controller.ts 106 lines

Handler	Behaviour
`register`	Validates email + 8-char password; rejects a duplicate with 409; bcrypt-hashes at cost 10; sets the refresh cookie and returns the access token plus a public user object. 201.
`login`	Compares against a **dummy hash when the user is missing**, so response time does not reveal whether an address is registered. Returns a single "Incorrect email or password".
`refresh`	Verifies the httpOnly cookie, re-checks the user still exists (they may have been deleted since issue), rotates both tokens.
`logout / me`	Clears the cookie; returns the current user.

The refresh cookie is `httpOnly`, `secure` in production, `sameSite: 'none'` in production (so a Vercel frontend on another domain can stay signed in) and `'lax'` locally, with a 7-day max age.

backend/src/controllers/session.controller.ts 471 lines

Session CRUD

- `getConfigOptions` — provider availability, the four personalities, fifteen languages and five experience levels, so the form is driven by the server.
- `createSession` — validates (JD ≥ 30 chars, ≥ 1 skill, duration 10–180, an explicit meeting provider), **creates the real meeting first** so a provider failure is a 400 with nothing written, then persists the session and kicks off question generation *without awaiting it* — several seconds of LLM work the recruiter should not wait for.
- `listSessions` — search across title and JD, filters on status / type / date range, pagination capped at 100, and per-row candidate, completed and question counts.
- `getSession` — the full session with candidates, their reports, submission counts, the ordered question set, and a per-candidate `joinUrl`.
- `updateSession` — a field-by-field patch; if `scheduledAt` actually moved, calls `SchedulerService.reschedule()`.
- `cancelSession` / `deleteSession` — cancelling also skips pending reminders and marks `INVITED` / `JOINED` candidates `CANCELLED`, so the platform stops chasing people for an interview that will not happen.

Question editing

`generateQuestions`, `addQuestion`, `updateQuestion` and `deleteQuestion` all call `assertNotStarted()` first.

> **WHY EDITING IS REFUSED ONCE THE INTERVIEW BEGINS**
>
> The state machine reads its questions when the interview starts and works from that list. Editing afterwards changes what the report is written against but not what was actually asked — the sort of mismatch nobody discovers until they are defending a hiring decision.

The coding payload has its own explicit Zod schema (`title, constraints[], starterCode, optimalTime, optimalSpace, testCases[{input, output, hidden}]`) rather than being passed through as free-form JSON, because that object is what the runner grades against — a mistyped field name would otherwise produce an exercise that silently fails every submission. Patches **merge** into `meta` rather than replacing it, since the editor sends only the fields it shows while the generator writes others (`redFlags` , `expectedPoints`) that nothing in the UI would put back.

Ranking

`getLeaderboard` , `getShortlist` (limit capped at 20) and `compareCandidates` (2–6 ids, every one confirmed to belong to a session this user owns).

backend/src/controllers/candidate.controller.ts 233 lines

- `attach()` — the shared path: upsert the `Candidate` by email (keeping the newest name, and only overwriting the mobile when one was supplied), reject a duplicate in this session with 409, create the `SessionCandidate` , and queue the invite plus three reminders.
- `bulkUploadCandidates` — CSV via PapaParse or XLSX via SheetJS, up to 1000 rows. `readRow()` normalises headings by lowercasing and stripping spaces, underscores and hyphens, so "Candidate Name", "full_name" and "NAME" all resolve. Each row is attempted independently and the response reports `inserted` , `skipped` and a per-row error list with the spreadsheet row number (offset by 2 for the header and 1-indexing).
- `listSessionCandidates` — omits `resumeText` , `resumeProfile` and `resumeQuestions` , which are large and never needed in a list view.
- `resendInvite` — upserts the `INVITE` reminder back to `PENDING` with attempts reset, then fires `SchedulerService.tick()` immediately rather than waiting up to a minute.
- `listAllCandidates` — the cross-session directory, with search and pagination.

backend/src/controllers/interview.controller.ts 449 lines

Both the candidate-facing token endpoints and the recruiter-facing inspection endpoints.

Handler	Behaviour
`getInterviewContext`	Everything the room needs before joining — candidate, session settings, interviewer persona, and the `JoinGate` verdict. **Deliberately excludes the question list**: the AI reveals questions one at a time.
`runCode`	Executes a submission. A `dryRun` uses visible cases only and is *not* recorded — it is feedback, not a graded attempt. A graded run includes hidden cases, stores the submission with the full execution detail, review and scores, and notifies any running meeting bot via `MeetBotManager.notifyCodingSubmitted()` . The response filters hidden cases out, returning only aggregate `hiddenPassed / hiddenTotal` .
`getCodingChallenge`	Serves the exercise to a candidate in a meeting call, who has no interview socket. Picks the first unanswered question, or the last one read-only if all are done. Hidden test cases never reach the browser.
`getCodingHint`	A nudge from `CodeAnalysisService.hint()` , logged to the transcript as a `SYSTEM` turn so the recruiter can see it was requested.
`postVideoMetrics`	Accepts 1–120 frames of four numbers each, each in 0..1, and stores the batch. Returns `{ recorded: 0 }` when video analysis is off for the session.

Handler	Behaviour
`uploadResume`	Candidate-side upload; refuses once the interview is closed. Reports whether AI analysis succeeded, with `analysisPending` when only the file and text were stored.
`getCandidateResume / downloadResume / uploadResumeForCandidate`	Recruiter-side: the parsed profile, the original bytes served straight out of Postgres with a sanitised filename, and an out-of-band upload path.
`getLiveInsights`	Insight list, aggregate summary and video summary in one round trip.
`evaluateInterview`	Clears the backoff first (so a manual retry is never blocked by exhausted automatic attempts) then runs evaluation synchronously. Safe to call repeatedly — it replaces the existing report.
`getEvaluationStatus`	Resolves to `READY`, `NOT_INTERVIEWED`, `NO_TRANSCRIPT`, `RETRYING`, `FAILED` or `PENDING`, plus attempts, error and next retry time.

backend/src/controllers/meetInterview.controller.ts — 759 lines — the largest controller

The recruiter API for interviews the AI runs inside a meeting the recruiter already created. The form is deliberately one screen — candidate, role, link, time — but each submission produces the same objects the rest of the platform works with: a session, a candidate, a question set and a transcript. Reports, analytics and exports therefore need no special-casing anywhere.

Booking

`bookInterview()` is the single path used by both the single form and every row of an import. It parses and validates the meeting link **before anything is written** (a mistyped link should surface while the form is open, not at the scheduled time), upserts the candidate, creates the session with `videoAnalysisEnabled: false` (a meeting has no client to capture from), creates the `MeetBotRun` with `joinAt = scheduledAt − lead`, kicks off question generation without awaiting, queues reminders when `sendInvite` is set, and finally calls `startIfDue()` then `armNextLaunch()`.

> **WHY BULK REUSES THE SINGLE PATH**
>
> A second, simpler path for bulk is how bulk-created interviews end up quietly missing their reminders or their questions.

Bulk import

- `downloadImportTemplate` — the CSV template, prefixed with a BOM because without one Excel opens a UTF-8 CSV as Latin-1 and turns every non-ASCII name into mojibake.
- `previewCandidateImport` — parses the file and returns rows *with their problems attached*. Nothing is written. The common failure is one bad cell in fifty good rows, and re-exporting the whole spreadsheet to fix it is miserable.
- `bulkCreateMeetInterviews` — books rows **in sequence**, not in parallel: each starts an LLM job and arms a bot launch, and fifty at once is a stampede. Partial success is the point — 48 good rows must not be thrown away because two had a typo. Capped at `MAX_IMPORT_ROWS = 200`, a limit set by the LLM quota rather than the database.
- `resolveRowOverrides()` — merges per-row values over the batch defaults through the same normalisers the preview used, so "behavioural", "45 mins" and "yes" mean here exactly what the preview said they meant.

Reading and control

`present()` is the single shape the dashboard renders. `evaluationState()` distinguishes seven outcomes — including `NOT_SCORED_INCOMPLETE` — each with a plain-English explanation.

> **WHY SEVEN STATES RATHER THAN "NO REPORT"**
>
> "No report" covers a candidate who never showed up, an interview that finished thirty seconds ago, and a provider quota that will clear in twenty minutes. Those need three different reactions from the recruiter.

`codingPageReachable()` probes `${APP_URL}/interview/reachability-probe/code` with a deliberately meaningless token — a missing route 404s for every token, a present route renders for any — and caches the answer for 60 seconds. It is checked while the recruiter is still looking at the form, because finding out mid-interview that the editor 404s cannot be recovered from.

`startMeetInterview` maps capacity and "already running" to **409**, since those are states of the world rather than bad requests. `getMeetInterviewStatus` is deliberately small — it is polled as a fallback whenever the socket is down — and includes `elapsedSeconds` so the dashboard clock needs no second request. `getMeetInterviewTranscript` numbers AI turns that carry a `questionId`, so a recruiter reading the transcript can line it up with the question set.

> **backend/src/controllers/report.controller.ts** 172 lines

`listRecentReports`, `getReport` (with scores pre-grouped by category so the UI does not have to), `getReportByCandidate`, PDF / Excel / session-Excel downloads, `sendFeedbackEmail` (refuses a second send without `force`, and stamps `feedbackEmailSentAt`), `updateRecommendation` (a human override) and `syncReportToAts`.

> **backend/src/controllers/analytics.controller.ts** 194 lines

- `getOverview` — eight counts in one `Promise.all`, average scores, a recommendation distribution, completion / no-show / hire rates, and a 14-day activity trend bucketed **in memory** to avoid 14 round-trips.
- `getSkillAnalytics` — reads the flat `Score` rows of category *Skill*, averages by normalised label, and returns the full list plus the strongest and weakest five.
- `getSessionAnalytics` — the last 20 sessions with candidate, completed, absent and evaluated counts, average overall and hire count.

> **backend/src/controllers/ats.controller.ts** 57 lines

List, create (URL-validated), delete, read the last 50 sync logs, and `testIntegration`, which fires the integration against the most recent report so the user can verify the receiving end before an interview depends on it.

5 · Backend — domain services

5.1 The interview engine

backend/src/services/InterviewStateMachine.ts — 969 lines · the heart of the system

Drives a single candidate's interview from joining through to completion. It owns the **script**; `LiveInterviewerService` owns the **conversation**. It extends `EventEmitter`, which is what lets the same class serve both the browser room (events → Socket.IO) and the meeting bot (events → synthesised speech).

States and events

```
States   WAITING_FOR_CANDIDATE → GREETING → IDENTITY_VERIFICATION
         → IN_ROUND ⇄ CODING → CLOSING → COMPLETED
                                      ↘ INCOMPLETE   (candidate left)
                                      ↘ ABSENT       (nobody joined)

Events   say      { text, questionId?, round?, expectsAnswer }
         state    { state, round?, progress }
         insight  { type, message, severity }
         coding   { question }
         thinking{ active }
         ended    { reason: completed | no_show | abandoned | ended_early }
```

Loading

`load()` pulls the session, the candidate and the ordered question set, then **materialises the candidate's resume questions** as synthetic `Question` rows with ids `resume-0…n` and order `10000+i`, placed in the `PROJECT` round. They are not persisted as real rows because the question set is shared by every candidate in the session. Everything is then sorted by `ROUND_ORDER` — `INTRO, HR, TECHNICAL, SCENARIO, PROJECT, CODING` — so the interview always flows in that shape regardless of insertion order, and `CODING` is filtered out entirely when the session has it disabled.

A hard-stop timer fires at **1.5×** the scheduled duration, protecting against a candidate who never stops talking and against a stuck client.

Localisation warm-up

`scriptLines()` enumerates every fixed line the interview can speak, and `localizer.warm()` translates them all while nothing is waiting. `prewarmScript()` is a static entry point the meeting bot calls *from its waiting room*, minutes before the state machine exists — both write into the same global cache.

> **WHY PREWARMING WAS NEEDED**
>
> The conversational turns arrive already localised because their system prompt names the language. Everything written as a string literal in the code is English. Without the warm-up, the very first Hindi interview translated its greeting at the exact moment it needed to speak it — and a provider blip at that moment meant the candidate was greeted in English.

The greeting exchange

People do not open with an agenda. The interviewer says `"Hi <first name>, how are you?"` and stops. `afterGreeting()` then classifies the reply into **unwell**, **asked-back**, **well** or **neutral** and answers it before moving on, capped at two turns of small talk.

WHY THE SPLIT MATTERS

A candidate who admits to being nervous and is told "glad to hear it" is worse off than one who got a recorded announcement. Sailing past a question the candidate asked you is the single most robotic thing an interviewer can do.

Silence handling — two different kinds

Method	Behaviour
`silenceDetected()`	Silence *on a question*. Records a `LONG_PAUSE` insight and offers to rephrase. In `CLOSING`, silence is treated as "no, I have no questions" and the interview finishes — offering to rephrase there is both wrong and a strange note to end on.
`greetingUnanswered()`	Silence *before a single word has been said*. Nothing has been asked yet, so this must not be scored as a skipped answer. The interviewer prompts up to `MAX_GREETING_PROMPTS = 4` times with escalating wording, then stops honestly as `abandoned` rather than walking the whole script past someone whose microphone was not working.

Answer handling

`afterAnswer()` logs the turn, asks `LiveInterviewerService.respond()` for a decision, runs `analyseAnswer()` for live signals, then applies the machine's own limits:

- **PROBE** — at most two follow-ups per question, so one question cannot eat the interview.
- **REPEAT / CLARIFY** — exactly one. A candidate who could not engage with the question the second time is not going to answer it on a third pass, and dwelling there burns their remaining questions.
- **END_EARLY** — closes properly and is still scored, because the interviewer chose to stop.
- **NEXT** — the acknowledgement is prefixed onto the next question.

A SUBTLE BUG THIS CODE EXISTS TO PREVENT

When the probe budget runs out, `spokenResponse` is still a follow-up *question*, and `advance()` puts the next question straight after whatever it is given. Passing it through asks two things in one breath — "...can you tell me something you are proud of? What is the difference between a controlled and an uncontrolled component in React?" — and the candidate answers neither. Only a `NEXT` response has been trimmed to a few words, so anything else is dropped for a plain "Thank you."

advance()

Resets the probe and repeat counters, increments the index, and normalises the acknowledgement — appending a full stop when the model omitted one, because joined onto the next question a missing stop reads as one run-on sentence to a voice engine (the regex covers the Hindi danda and the Arabic question mark). When questions run out it moves to `CLOSING`; on a coding question it emits the `coding` event with hidden test cases stripped and speaks a wrapper around the problem text.

The closing exchange

`afterClosing()` has three outcomes rather than one: a refusal closes, a **bare affirmative** ("Yes.") invites the question and waits (bounded at two prompts), and anything else is treated as the question itself and answered by `answerClosingQuestion()`.

> **WHY**
>
> This used to end the interview on whatever came back, which meant an eager "Yes." was answered with goodbye — the interviewer asked a question and then hung up before the candidate could ask theirs. `isBareAffirmative()` is deliberately narrow (≤ 5 words, purely affirmative), because speech-to-text seldom supplies the question mark that would settle it and a candidate who says "yes, what does the team look like" has already asked.

finish()

Only an interview the AI carried through to its closing round becomes `COMPLETED` and earns a score. A candidate who cuts it short is recorded as `INCOMPLETE` — a partial interview cannot be scored fairly against a full one, and a low score caused by walking out would misrepresent them. The transcript is kept in full either way. `concludeEarly()` lets the meeting bot wrap up on time by jumping to the closing round; it waits up to five seconds for an in-flight turn rather than talking over it.

> **backend/src/services/LiveInterviewerService.ts** 393 lines

Holds one candidate's conversation with the AI. Every call returns a JSON object validated against `turnSchema`: `{ decision, spokenResponse, answerQuality, note }`.

The system prompt

Structured into named blocks: **HOW TO SOUND** (contractions, short sentences, no em dashes or lists — none of them exist out loud), **SESSION**, **PERSONALITY**, an optional **RESUME** block, **SPEECH RULES** (two sentences max, exactly one question per turn), **HARD RULES**, **DECISIONS** and **SAFETY**.

Per-round guidance is injected from `ROUND_GUIDANCE` — for example CODING says "read the problem aloud once, then stay quiet while they work; never review their code or comment on correctness."

The rule that outranks every other

> **NEVER HAND THE CANDIDATE THE ANSWER**
>
> If the candidate asks the question back, asks what the tested concept means, or asks the interviewer to answer it — *that is the test*. The prompt forbids explaining anything, and `OWN_WORDS_LINE` is the fixed reply. Because a small model will still occasionally leak the content no matter what the prompt says, `sanitise()` enforces it in code.

sanitise() — three code-level guards

1. **Unprompted END_EARLY** is overridden to `NEXT` unless the candidate's own words match a stop/unwell/leave pattern, and the override is recorded in the private note. Otherwise the interview would be cut short and the remaining questions lost.
2. **A REPEAT or CLARIFY with no question mark** is almost always the model explaining the tested content instead of re-asking, so it is replaced with `OWN_WORDS_LINE`.
3. **A NEXT acknowledgement** is truncated to its first sentence, and dropped entirely for "Thank you." if it exceeds eight words or reintroduces the interviewer — otherwise the candidate hears the interviewer introduce itself twice.

Other entry points

- `greeting()` — one line, using the candidate's first name only. "Hi Priyanshu Raj" is how a form addresses you, not how someone opening a conversation does.
- `intro(opener)` — the handover from small talk into the interview, prefixed with the reply to whatever they just said so it reads as one continuous turn.

- `handleDoubt()` — a mid-question clarification, with the same no-question-mark guard.
- `answerClosingQuestion()` — deliberately separate from `handleDoubt`, which withholds answers on purpose. There is no question on the table and this is the one moment the candidate is owed a straight answer. The prompt states plainly what the interviewer does *not* know (salary, start date, team size, benefits, timelines) so it says the recruiting team will confirm rather than inventing a number.
- `trimHistory()` — keeps the system prompt plus the last 24 turns, so long interviews do not blow the context window.
- Any model failure falls back to a neutral "Understood, thank you." with `decision: NEXT` — the interview never stalls on a bad turn.

backend/src/services/personality.ts 73 lines

Key	Name	Tone	Probe bias
FRIENDLY	Aria	Warm and encouraging; puts nervous candidates at ease	0.35
NEUTRAL	Ravi	Even and professional; steady pace	0.50
FORMAL	Mr. Sharma	Structured and precise; suits regulated or senior roles	0.50
CHALLENGING	Dr. Rao	Direct and demanding, never rude; presses on vague claims	0.75

Fifteen locales: English (US / India / GB), Hindi, Spanish, French, German, Portuguese (BR), Japanese, Mandarin, Arabic, Tamil, Telugu, Bengali and Marathi.

backend/src/services/localize.ts 174 lines

Translates the interview's scripted skeleton into the session's language, one line at a time, cached globally by `(language, masked text)`. English sessions never reach the model at all.

> **WHY NAMES ARE MASKED OUT**
>
> Left to the model, "Priyanshu" came back as a mix of Devanagari and Bengali glyphs, and a mangled name in the greeting is the worst possible first impression. Protected terms become `{{P0}}` placeholders before the model sees the line and are restored afterwards. Masking is also what makes the cache work across candidates: the greeting differs only by name, so with the name masked every interview shares one cached translation.

Two attempts with a 2.5-second pause between them, using the **smart** model — each line is translated once per language ever, so quality is worth far more than speed. The fast model demonstrably was not up to it: it turned "the role you have applied for" into the interviewer claiming to have applied. A translation dramatically shorter than the source, or one missing a placeholder, is rejected and retried. Markdown emphasis is stripped because a voice reads it out as noise. After two failures the English line is spoken — an interviewer that momentarily switches to English is a flaw; one that goes silent is a dead interview.

5.2 Questions, resumes and evaluation

backend/src/services/QuestionGenerationService.ts 421 lines

planQuestionMix()

Decides how many questions of each kind fit the interview. A coding task is budgeted at 6–10 minutes but never more than 45 % of the round; the remaining minutes minus 3 (intro + closing) are divided into 2.5-minute slots. The mix then differs by type:

Type	Composition
HR	1 intro, most slots behavioural, 1 scenario, 1 project, no coding
TECHNICAL	1 intro, ~55 % technical, ~20 % scenario, ~20 % project, 1 coding
MIXED	1 intro, 30 % HR, 35 % technical, 15 % scenario, 15 % project, 1 coding

> **CODING IS NEVER DROPPED FOR BEING "TOO SHORT"**
>
> Silently ignoring an explicitly enabled feature is worse than a tight schedule. A short interview gets a smaller coding budget and fewer spoken questions instead.

The prompt

Names the role, JD, skills, seniority, type, duration and language, states the exact count per category, then imposes rules: no greeting or preamble inside a question (the interviewer has already introduced itself), every technical question must map to a named skill, calibrate to the stated level (no system design for freshers, no syntax trivia for seniors), nothing answerable in one word, no two questions testing the same thing, written the way a person says them out loud, and grounded in the JD's actual domain.

Coding problem rules — strict, because a broken problem wastes the interview

- Grading runs the program and compares **stdout**. There is no function harness and no return value, so the prompt forbids "write a function that takes X and returns Y".
- Phrased as "read <this> from standard input, print <that> to standard output", with the exact line-by-line input and output format inside the question text.
- 4–6 test cases, at least 2 marked `hidden` so the visible answers cannot be hard-coded.
- No linked lists, trees or any structure that cannot be expressed as plain lines of text.

repairCodingQuestions()

Coding problems are the easiest thing for a model to get subtly wrong, so generated ones are validated and anything unusable is **replaced with the known-good template** rather than shipped. A question is dropped when it has fewer than two runnable cases, when its wording matches the function-style regex, or when it mentions an unsupported structure. If no case is hidden, the back half is marked hidden automatically.

Fallback and persistence

`fallbackSet()` is a deterministic template set — five behavioural questions, per-skill technical and scenario templates, a project question and a fully specified Two Sum with correct test cases — so an LLM outage never leaves a session unusable. `save()` deletes any existing set and writes the new one in a single `createMany`, stamping `generatedBy` with the model name and packing category-specific payloads into `meta`.

backend/src/services/ResumeService.ts 406 lines

Method	Behaviour
`extractText()`	PDF via `pdf-parse` v2 (imported lazily — it pulls in pdfjs and would slow boot; the worker is explicitly destroyed or it keeps the process alive), DOCX via `mammoth`, plain text directly.
`normalise()`	Collapses the whitespace soup PDF extraction produces and caps at 24,000 characters.
`analyse()`	Reads the resume *against the job description* into a structured profile. The prompt insists on never inferring: an empty list is correct when the resume does not say.
`buildQuestions()`	Writes 2–4 questions (scaled by duration) naming a real project, employer or claim, prioritising `claimsToProbe` and establishing the real level of any required skill the resume never evidenced.
`attach()`	Extract → **store** → analyse → build questions → store profile and questions.
`analysePending()`	Picks up one resume per scheduler tick whose file is stored but `resumeParsedAt` is still null.
`promptBlock()`	Condenses the profile into the block injected into the live interviewer's system prompt, ending with a 4,000-character excerpt so the model can quote accurately.

WHY THE FILE IS STORED BEFORE ANALYSIS IS ATTEMPTED

Analysis needs the LLM, and a provider outage must never cost the candidate their upload — they may not be able to re-upload before the interview starts. The bytes and extracted text go into Postgres first; leaving `resumeParsedAt` null is what marks the row for the scheduler to finish later.

The interesting output is not the skill list — it is `claimsToProbe` (3–6 quantified or senior-sounding claims worth verifying live, quoted recognisably) and `missingJdSkills` (required skills with no supporting evidence anywhere). A file yielding fewer than 120 characters is rejected with advice about scanned PDFs.

backend/src/services/EvaluationService.ts 411 lines

Produces the full report for one interview by combining four independent sources.

1 · Measured signals

`extractSignals()` + `scoreCommunication()` from `CommunicationAnalyzer`, the insight summary, and the video summary when enabled.

2 · Coding

Each submission scores `correctness × 0.6 + quality × 0.4`; the interview's coding score is the average, or `null` when there were no submissions.

3 · LLM judgement

The transcript is rendered as numbered Q/A pairs. The prompt supplies the role, JD, skills, seniority and — when a resume exists — what it claimed, with the explicit instruction to **judge the interview, not the resume**, using the resume only to notice a gap between what was claimed and what could actually be explained. Measured values are passed in as *inputs* with "do not re-estimate these". Scoring rules require every strength and weakness to cite a specific moment, allow `null` for any dimension with too little evidence ("do not guess and do not default to 5"), and reserve red flags for dishonesty, contradiction, hostility or a total absence of claimed expertise.

4 · Composition

```
technicalScore   = mean of  knowledge, problemSolving, logicalThinking,
                            projectUnderstanding, domainExpertise   (nulls dropped)
behavioralScore  = mean of  leadership, teamwork, adaptability,
                            ownership, learningMindset              (nulls dropped)
communicationScore = measured × 0.8  +  observed composure × 0.2

weights            technical  communication  behavioral  coding  video
  HR                  0.15         0.40         0.40      0.00    0.05
  TECHNICAL           0.20         0.15         0.10      0.50    0.05
  MIXED               0.15         0.20         0.10      0.50    0.05

overall = Σ(score × weight) / Σ(weight of dimensions actually measured)
```

> **WHY WEIGHT IS REDISTRIBUTED RATHER THAN ZEROED**
>
> A dimension that could not be measured is not evidence of a zero. Dividing by the usable weight keeps an unmeasured dimension from silently dragging the score down.

Hard gates on the recommendation

- ≥ 3 red flags → **REJECT**, regardless of score.
- overall < passMark − 2 → **REJECT**; overall < passMark → **CONSIDER**.
- **STRONG_HIRE** additionally requires overall ≥ 8.5, technical ≥ 8, communication ≥ 7 and zero red flags — so a fluent talker with no substance cannot top the list.
- overall ≥ passMark + 1 with technical ≥ passMark → **HIRE**; otherwise **CONSIDER**.

Persistence

Everything is written in one transaction: delete any prior report, create the new one with the full `details` blob, then `createMany` the flat `Score` rows — Overall, the six communication sub-scores, each non-null technical and behavioural sub-score, one row per discussed skill with its evidence, and coding / video summaries. The LLM summary is copied onto the transcript so the recruiter list is scannable.

`PermanentEvaluationError` is thrown when there are no candidate responses at all — no amount of retrying will help, so the queue must stop immediately rather than burn five attempts on an empty transcript.

> **backend/src/services/CommunicationAnalyzer.ts** 206 lines

Pure functions, no I/O, no model. `extractSignals()` walks the candidate's turns and computes seventeen measurements: word totals, average answer length, average and maximum latency, count of pauses over 5 s, filler and hedge counts and ratios, words per minute, lexical richness (unique content words ÷ total content words, stop words excluded), average ASR confidence, sentence count and length, and a repetition ratio.

`scoreCommunication()` maps those onto six 0–10 sub-scores using `scoreBand()`, a piecewise-linear ideal band with a "too high" arm:

Sub-score	Composition
Fluency	50 % answer-length band (ideal 45–160 words) + 50 % filler penalty
Confidence	40 % latency band (ideal ≤ 1.5 s) + 30 % long-pause penalty + 30 % hedging penalty
Clarity	45 % ASR confidence + 35 % sentence-length band (ideal 12–25 words) + 20 % repetition penalty

Sub-score	Composition
Grammar	50 % sentence-structure regularity + 50 % filler-free phrasing
Vocabulary	75 % lexical richness band + 25 % repetition penalty
Pace	Words-per-minute band; 110–165 wpm reads as comfortable conversational speech

Overall is a weighted blend (fluency .22, confidence .22, clarity .20, grammar .14, vocabulary .12, pace .10). Human-readable notes are generated alongside — "3 pauses over 5 seconds before answering", "spoke quickly at 195 words per minute".

> **WHY THIS IS RULE-BASED**
>
> The communication score is reproducible and defensible. A recruiter can be shown the measurement behind it, and it will be the same number tomorrow — which an LLM opinion would not be.

backend/src/services/InsightService.ts 196 lines

`analyseAnswer()` derives real-time signals from a single answer, instantly and without a model call — the interview loop cannot wait on an LLM to decide whether the candidate hesitated. It fires on: latency over 8 s (`LONG_PAUSE`) or 4 s (`HESITATION`), filler density above 8 % in a 15-word-plus answer, explicit "I don't know" or three hedges (`LOW_CONFIDENCE`), answers under 6 words, ASR confidence below 0.55, speaking pace above 190 or below 85 wpm, and a `STRONG_ANSWER` when quality ≥ 0.75 with 40+ words, low fillers and at most one hedge.

`summarise()` aggregates into counts, positive and negative totals, and a 0–10 **composure** score that penalises accumulated severity and rewards strong answers. That score becomes 20 % of the final communication figure.

backend/src/services/TranscriptService.ts 87 lines

`ensure()` creates the transcript on first use and survives a race with a concurrent turn by re-reading on conflict. `logTurn()` computes words-per-minute for candidate turns where the client measured duration. `format()` renders plain text for prompting and export; `toQnA()` pairs each AI question with the reply that followed, which is the shape the evaluator reads.

backend/src/services/VideoAnalysisService.ts 193 lines

`computeFrameConfidence()` = `presence×0.45 + stability×0.45 − motionPenalty×0.25 + 0.05` , where the penalty only applies above 0.45 motion because a little movement is natural. `inferExpression()` labels a window `ABSENT` , `RESTLESS` , `ENGAGED` , `DISTRACTED` , `TENSE` or `NEUTRAL` . `summarise()` returns averages, a dominant expression, an expression histogram, a 0–10 confidence and engagement score, and plain-English observations. `timeline()` buckets frames into at most 24 points so a long interview still renders as a readable chart.

> **HONEST LABELLING**
>
> This is derived from skin-tone centroid stability and inter-frame motion, not a trained facial-expression model. The labels are honest proxies for presence and steadiness — engagement signals, not psychology.

5.3 Coding, scheduling and the join gate

backend/src/services/CodeExecutorService.ts 295 lines

Runs submitted code as a child process against the question's test cases. Four languages, resolved through an alias table: JavaScript / Node / TypeScript, Python, C++, Java.

Defence in depth

Control	Detail
Static ban list	Regexes rejecting `child_process`, `subprocess`, `socket`, `shutil`, `require('fs'|'net'|'http'|…)`, `import os`, `os.system`, `__import__`, `Runtime.getRuntime`, `ProcessBuilder`, `java.io.File`, `fopen` / `popen`. Reading stdin is deliberately *not* banned.
Blank environment	The child gets only `PATH` and `SYSTEMROOT`, keeping credentials out of reach of submitted code.
Temp working directory	A UUID-named directory under the OS temp dir, removed in a `finally` block.
Wall-clock limit	`CODE_EXEC_TIMEOUT_MS` per case (default 5 s), enforced with `SIGKILL`; 30 s for a compile.
Output cap	256 KB per stream, so a runaway print loop cannot exhaust memory.
Memory caps	`javac -J-Xmx256m` and `java -Xmx256m`; `g++` compiles **without** `-O2`.
Kill switch	`CODE_EXEC_ENABLED=false` returns an `unsupported` result rather than running anything.

WHY NO -O2

Optimising an interview answer buys nothing measurable and costs a great deal: with `<bits/stdc++.h>` it is the difference between a compiler needing a couple of hundred megabytes and one needing most of a gigabyte. On a box that is also running a browser in a live meeting, that is what gets the compiler killed.

The JavaScript harness

Node has no built-in synchronous stdin reader and the usual idiom needs `fs`, which the ban list blocks. A prelude injected ahead of the candidate's code reads stdin once and hands them `input`, `lines`, `readline()` and `readInts()` with no imports at all. Because that shifts line numbers, `realignJsStack()` rewrites `main.js:<n>` in stack traces back to what the candidate actually wrote.

Comparison and diagnostics

`outputMatches()` normalises CRLF, trims each line's trailing whitespace and trims the whole string — models and humans disagree on trailing newlines and it should not decide a hire. `describeCompileFailure()` exists because a compiler killed by the OOM killer exits non-zero with *empty stderr*, indistinguishable from a compiler that is not installed. It only blames a missing compiler when the process could not be spawned at all (`exitCode === -1`), and otherwise says the server most likely ran out of memory — "this is a server problem, not a problem with your code."

backend/src/services/CodeAnalysisService.ts 158 lines

`staticMetrics()` computes lines, comment ratio, maximum nesting, loop and branch counts and a rough cyclomatic complexity without a model. `review()` asks the smart model to judge the code as written — the actual complexity of *their* algorithm, not the optimal one — and is told that code failing its tests cannot score above 4 for quality. Only

failure *shapes* are sent to the model (timed out / errored / wrong output), never hidden expected outputs. A model failure falls back to metrics-derived values rather than fabricated ones. `hint()` returns a single subtle nudge that never writes code or names the algorithm. `score()` blends `correctness×0.6 + quality×0.2 + complexityBonus×0.1 + readability×0.1`.

backend/src/services/JoinGate.ts 104 lines

One pure function, `evaluateJoinGate()`, returning `{ canJoin, verdict, reason, opensAt, expiresAt }`.

```
        scheduledAt              +grace/2            +grace
─────────────┼──────────────────────────┼──────────────────┼──────────────
  TOO_EARLY  │          OPEN            │      OPEN        │   EXPIRED
             │                          │ nudge email      │ marked ABSENT
```

- The room does **not** open early; a 60-second tolerance absorbs clock skew so a punctual candidate is never told they are early.
- An unused link expires at `scheduledAt + NO_SHOW_GRACE_MINUTES`, matching exactly when the scheduler marks the candidate absent.
- A candidate who *has* joined keeps their link regardless of the clock, so a dropped connection mid-interview can always be resumed.
- Cancelled, completed and absent each get their own verdict and candidate-facing sentence.

> **WHY ONE FUNCTION, CALLED TWICE**
>
> The same rule is enforced in the HTTP context endpoint *and* in the Socket.IO handshake, because the socket is reachable directly and an HTTP-only check would be advisory. Both call this; there is no second copy of the logic that could drift.

backend/src/services/SchedulerService.ts 385 lines

A single interval (default 60 s, disabled entirely by `SCHEDULER_ENABLED=false`) with a re-entrancy guard so a slow tick cannot overlap the next one.

One tick, in order

1. `sendDueReminders()` — up to 50 pending reminders whose moment has passed and which have fewer than 3 attempts. Cancelled sessions and terminal candidates are marked `SKIPPED` rather than mailed. Failures increment attempts and become `FAILED` at 3.
2. `activateDueSessions()` — flips `SCHEDULED` to `ACTIVE` at the start time.
3. `MeetBotManager.startDue()` — launches bots whose lead time has arrived. Runs *before* the no-show sweep so a bot is already waiting by the time anyone is considered late.
4. `sendNoShowNudges()` — halfway through the grace window, for candidates who never opened their link. Fires once per candidate, enforced by the unique `(sessionCandidate, kind)` reminder row; a failure is still recorded so a broken mailbox is not retried forever.
5. `markNoShows()` — marks absent after the grace period. **Skips interviews whose bot is still waiting**, using the bot's own clock (wait + grace), because writing the candidate off here would end an interview still sitting in the meeting waiting for them.
6. `closeFinishedSessions()` — closes sessions where every candidate has reached a terminal state.
7. `EvaluationQueue.processPending()` — one outstanding report.

8. `ResumeService.analysePending()` — one unanalysed resume.

`scheduleFor()` queues the invite immediately plus T-24 h / T-1 h / T-5 min, marking any whose moment already passed as `SKIPPED` — so adding a candidate an hour before the interview does not spam them with three back-dated reminders. `reschedule()` re-times pending reminders *and* the bot's `joinAt`, then re-arms the launch timer.

> **WHY THE BOT LAUNCH MUST MOVE TOO**
>
> Without it a rescheduled session keeps its original `joinAt`, so the bot either turns up to a meeting that is not happening yet or — far more often — is written off as a missed window and never joins at all.

`joinUrlFor()` sends a meeting-interview candidate to the *meeting*, not the built-in room, because the interviewer is not in the built-in room. `codingUrlFor()` withholds the coding link from the invitation by default: sending it ahead of time hands the candidate the exercise before the interviewer has asked for it, which is the wrong order.

backend/src/services/EvaluationQueue.ts 142 lines

Makes report generation self-healing. A candidate can finish a full interview and still get no report if the LLM is rate limited or the database blips at the wrong moment; losing the whole assessment to a transient failure is unacceptable, so a completed interview without a report is treated as outstanding work.

- `run()` never throws — callers are fire-and-forget paths such as the socket gateway.
- A `PermanentEvaluationError` jumps straight to `MAX_ATTEMPTS` and clears the retry time. Nothing to score means nothing will ever be scored.
- A rate limit uses the provider's own hint plus 30 s; anything else backs off exponentially — 60 s, 120 s, 240 s, capped at 30 minutes so retries stay same-day.
- `processPending()` returns immediately while a cooldown is active — no point querying, let alone calling the model, while the provider is known to be blocked. That is what produced the original retry flood.
- It takes **one** interview per tick, because evaluation is token-heavy and a burst is what caused the rate limit in the first place, and skips anything completed less than 45 seconds ago to let the gateway's own attempt land first.
- `INCOMPLETE` interviews are excluded from the query entirely.

5.4 Ranking, export and ATS

backend/src/services/RankingService.ts 146 lines

`rankSession()` orders by **hiring recommendation first**, then overall, then technical. A *Hire* always outranks a *Consider* even when the raw numbers are close, because the recommendation encodes the hard gates — red flags and the technical floor — that the number alone does not. Unevaluated candidates sink to the bottom with rank 0 and are sorted by name among themselves.

`compare()` returns the selected candidates with scores pre-grouped by category so the UI can render one radar per dimension without regrouping. `shortlist()` takes Strong Hire and Hire; if there are none it falls back to Consider and says so in an honest note — "No candidate reached Hire. These are the strongest of those worth a second look."

backend/src/services/ExportService.ts 477 lines

reportPdf()

Streams a PDFKit A4 document straight to the response. Layout helpers (`rule` , `heading` , `bullet`) handle page breaks by checking remaining height before writing. Sections: branded header, candidate identity, a facts table (role, type, level, date, duration, questions asked, identity verified), the recommendation banner with its reason, scores, then strengths and weaknesses with their evidence, improvements, red flags, skill breakdown, coding results, video observations and the transcript.

The verdict is presented in two halves above the individual dimensions: a **soft** average of communication and behavioural, and a **hard** average of technical and coding (technical alone when there was no coding round).

reportExcel() and sessionExcel()

ExcelJS workbooks with styled header rows. The report workbook has Overview, Scores (category / metric / score / out-of / evidence), Feedback (strengths, weaknesses, improvements, red flags), Coding and Transcript sheets. The session workbook has a Session info sheet and a Candidates sheet — rank, contact, status, every score, recommendation and the top strength and weakness — driven by `RankingService.rankSession()` so the spreadsheet order matches the leaderboard.

backend/src/services/AtsService.ts 199 lines

`buildPayload()` produces one normalised `interview.completed` body — candidate, job, interview metadata, all five scores, recommendation and reason, summary, strengths, weaknesses and a deep link to the report — identical for every provider, so a generic webhook receiver works everywhere. Only the auth header differs: Greenhouse gets HTTP Basic (`key:` base64-encoded), everyone else a bearer token. Every attempt is logged to `AtsSyncLog` with the status and any error body, **including failures**, and a 15-second timeout bounds the call. `syncAll()` fans out with `Promise.allSettled` so one broken integration cannot hide the others' results. Stored API keys are never returned by any read path.

backend/src/services/CandidateImportService.ts 578 lines

Turns a recruiter's spreadsheet into rows the booking endpoint accepts. The design principle is stated in the file: **forgiving about shape, strict about content**.

- **Column aliases** — headings are squashed (lowercased, non-alphanumerics removed) and matched against an ordered alias list, so "Mobile Number", "mobile_number" and "Mobile-No." all collapse to the same key. Order matters: exact names come first so `name` does not swallow "Company Name". Fourteen fields are recognised, including per-row overrides for job title, description, skills, duration, type, personality, language and coding.
- **Split date and time columns** are merged via `TIME_ALIASES` and `joinDateAndTime()` .
- **Excel float rounding** — `toMinute()` rounds to the nearest minute, because Excel stores a time as a fraction of a day and 11:15 comes back as 11:14:59.999. An invitation reading 11:14 for a meeting typed as 11:15 looks like the system got it wrong.
- **Normalisers** — `normalizeType` , `normalizePersonality` , `normalizeLanguage` , `normalizeExperienceLevel` , `parseDurationMinutes` , `parseYesNo` , `splitSkills` . The booking path calls the same ones, so the preview and the result cannot disagree.
- **Meeting links** are validated with `isSupportedMeetingLink()` at preview time.
- **Errors are per field, per row**, keyed so the UI can highlight the exact cell; blank rows are counted and dropped silently, and unmapped columns are returned so the recruiter can see what was ignored.

6 · Backend — realtime gateways

> Three Socket.IO namespaces on the same HTTP server. Each has a different audience and a different trust model.

backend/src/realtime/interviewGateway.ts — 344 lines · namespace /interview

The candidate's live interview room. A `Room` holds the state machine, the set of attached sockets, a disconnect timer, the Deepgram session, the turn start time, the listening gate and the interview language.

Connection

The access token arrives as a handshake query parameter. Resolving it needs a database round-trip, but the client may emit `candidate_joined` the instant it connects — and Socket.IO **drops events that arrive before a listener exists**. So every listener is registered synchronously and each one awaits the same `roomReady` promise before acting, through a `withRoom()` helper. A failed handshake emits an error and disconnects rather than leaving the candidate in silence.

`openRoom()` applies `evaluateJoinGate()` — the same gate as the HTTP endpoint — and on refusal emits `interview_closed` with the verdict and reason before disconnecting.

Protocol

Client → server	Effect
`candidate_joined`	Starts the machine, or replays the current position on a reconnect
`candidate_answer`	One utterance with `latencyMs`, `durationMs`, `confidence`
`candidate_doubt`	An out-of-band question mid-round
`silence_timeout`	Client-side silence detection
`coding_submitted`	Resumes the round after grading
`audio_chunk`	WebM/Opus audio forwarded to Deepgram
`listening_started / _stopped`	Opens and closes the answer gate, and stamps the turn start
`proctoring_event`	Tab switch, copy or paste — recorded as an insight with severity scaled by how long they were away
`end_interview`	The candidate left

Server → client	Effect
`connected`	Session id, current state, candidate name
`ai_speak`	What to say, whether an answer is expected
`state_change`	State, round, progress percentage
`ai_thinking`	Drives the thinking indicator
`insight`	A live behavioural signal
`coding_challenge`	The exercise, hidden cases stripped

Server → client	Effect
`interim_transcript / final_transcript`	What is being heard
`speech_unavailable`	Fall back to browser recognition
`interview_ended / report_ready`	Terminal events

Server-side speech

A `SpeechSession` is created lazily on the first audio chunk. Final fragments are accumulated until Deepgram signals `speech_final`, with `UtteranceEnd` as a backstop for a candidate trailing off mid-sentence. On flush, if the gate is closed the text is **discarded** — heard while the interviewer was speaking or thinking, it is almost certainly the AI's own voice coming back through the candidate's speakers. Otherwise the averaged confidence and measured latency are handed to the machine as an answer.

> **WHY DEEPGRAM DECIDES WHEN A TURN ENDS**
>
> Its end-of-speech signal beats a fixed client-side silence timer: it tolerates a candidate pausing mid-sentence to think.

Disconnect

When the last socket drops, a 45-second grace timer starts; if nobody returns, the machine is told the candidate left. On `ended`, the room is torn down, the speech session stopped, and — only for `completed` or `ended_early` — `EvaluationQueue.run()` is fired, emitting `report_ready` on success. Failures are recorded and retried by the scheduler rather than lost.

backend/src/realtime/meetBotGateway.ts — 132 lines · namespace /meet-bot

The recruiter's live window onto a running meeting interview. **Read-only by design**: this namespace reports what the bot is doing and never accepts instructions. Starting and stopping go through the authenticated REST endpoints, where ownership is checked properly and the action is recorded.

- Authenticated by JWT in the handshake, verified in middleware; a recruiter may only subscribe to interviews from their own sessions.
- `MeetBotManager.setEventSink()` is the single funnel — every session event is mapped to a wire name (`interview:status`, `:joined`, `:message`, `:question`, `:answer`, `:interim`, `:completed`, `:error`) and broadcast to that interview's room. A synthetic `interview:started` is derived from the first `STARTING` status, because that is a distinct moment recruiters watch for.
- A 60-event replay buffer per interview is emitted to a recruiter who opens the page late, after the current status. Interim transcripts are excluded — they are superseded within a second and would show stale half-sentences. The buffer is dropped a minute after a terminal status.

backend/src/realtime/codingGateway.ts — 109 lines · namespace /coding

Mirrors the candidate's editor into the meeting. A meeting call has no shared editor, so the candidate writes in their own tab — which leaves everyone else unable to see the work as it happens, and that is most of the value of a live coding round.

The editor broadcasts `code:update`; a read-only spectator page renders it; the bot opens that page in a second tab and presents it into the meeting. The room key is the interview and the credential is the same access token that opens the editor, so nothing new has to be issued or remembered. The latest state per interview is cached in memory so a

spectator joining halfway through does not stare at an empty editor; code is bounded at 60,000 characters and entries are pruned after 6 hours. Nothing here is a record — the transcript and the submissions are.

7 · Backend — the meeting bot

> Sixteen files under `services/meetingBot/`. A Playwright-driven Chromium joins a Google Meet, Zoom or Microsoft Teams call the recruiter already created, waits for the candidate, conducts the interview by voice, writes the transcript and files a report. It never creates a meeting, never types a password, and never tries to get past a security check.

7.1 Manager and session

backend/src/services/meetingBot/MeetBotManager.ts — 561 lines · static class

Two locks, because neither alone is enough

An in-memory `Map` catches a second request to *this* process. A conditional database update catches a second replica or an overlapping scheduler tick:

```
updateMany({
  where: { sessionCandidateId, status: { in: startable },
           OR: [ { lockedBy: null }, { lockedBy: INSTANCE_ID },
                 { lockedAt: { lt: staleBefore } } ] },
  data:  { lockedBy: INSTANCE_ID, lockedAt: now, status: 'STARTING' },
})   // count === 1  ⇒  we own it
```

`INSTANCE_ID` is `hostname:pid:uuid8`. A lock older than `STALE_LOCK_MS = 45 min` belonged to a process that is gone — comfortably longer than the slowest legitimate join (ten minutes in a lobby plus the candidate wait), so a live interview is never stolen.

Scheduling

- `startDue()` — called every scheduler tick; must be cheap when idle and must never throw, or it would take reminders and evaluations down with it. It re-arms the launch timer in a `finally`, never before the claims, or an overdue `SCHEDULED` run would arm a zero-delay timer that calls straight back in.
- `launchDue()` — takes runs whose `joinAt` falls within `MEET_BOT_WARMUP_SECONDS`. Cancelled sessions are closed; a run whose entire window elapsed while nothing was running (a restart during the slot) is recorded as a miss rather than joining a meeting long since over.
- `armNextLaunch()` — a `setTimeout` for the exact moment the next interview is due, floored at 1 s.

> **WHY A TIMER AS WELL AS THE TICK**
>
> The scheduler ticks once a minute, which was fine while the bot joined five minutes early. Joining *at* the scheduled time makes that minute the difference between punctual and visibly late. The tick remains the backstop — it re-arms the timer every pass, so a missed timer costs punctuality, never the interview.

> **WHY THE WARM-UP LEAD IS NOT "JOINING EARLY"**
>
> Everything before the join press happens on the pre-join screen, where the meeting cannot see the bot. The driver holds the press until the scheduled second, so the browser's startup time is bought back without the bot turning up early.

Other responsibilities

- `startIfDue()` — called the moment an interview is created, so scheduling one for "in two minutes" (or right now, which is what testing looks like) does not sit idle until the next tick. Waiting for a clock that has already struck is not scheduling, it is latency.
- `warmBrowserBinary()` — opens and closes a throwaway browser in its *own* temp directory (Chromium takes an exclusive lock on a user-data directory, and this must never collide with an interview). Measured: 40 s for the first launch after boot, 0.4 s after. That bill was being paid by whichever interview happened to be first after a deploy.
- `recoverOrphans()` — at boot, marks runs whose lock is stale or null as `FAILED / BROWSER_CRASHED` with "The server restarted while this interview was running."
- `notifyCodingSubmitted()` — how an HTTP submission from a tab that knows nothing about the meeting reaches the interviewer waiting in the call.
- `upcoming()` / `logUpcoming()` — the queue, printed at boot, because the commonest reason an interview "did not join automatically" is that nothing was running when its moment came.
- `shutdown()` — asks every running session to leave politely.

> **`backend/src/services/meetingBot/MeetBotSession.ts`** `1,055 lines · the largest file in the repository`

One interview, from launching a browser to filing the report. **The interview itself is not reimplemented here** — `InterviewStateMachine` owns the script and `LiveInterviewerService` owns the conversation, exactly as they do for the built-in room. This class is the transport: it turns the machine's `say` events into audio in the meeting, and Deepgram's transcripts into answers.

run()

1. `armLifetimeGuard()` — before anything can hang. A session occupies one of a small number of concurrency slots, so one that never finishes does not just lose its own interview, it silently stops every later one from launching. The budget is the sum of every configured wait plus 1.5× the duration plus 15 minutes.
2. `load()` — re-parses the stored link rather than trusting the stored platform label ("the stored platform is a label, the link is the fact"), resolves the driver, and reads schedule, duration, candidate name and coding URL.
3. `launchBrowser()` — isolated profile only when concurrency > 1; a signed-in profile required only for Google Meet.
4. `audio.install()` — **before navigation**, because the `getUserMedia` and `RTCPeerConnection` overrides must be in place before the meeting client's bundle captures references to the originals.
5. `joinWithRetries()` — three attempts, but only for retryable failures. A signed-out account, a denied admission or a link to nowhere throws on the first. The page is screenshotted and dumped on the attempt that gives up.
6. `verifyBridge()`, wire audio, start the `MeetingMonitor`, `prewarmScript()` for non-English sessions.
7. `waitForCandidate()`, then `startInterview()`.

waitForCandidate()

Two independent presence signals, because the first is not trustworthy alone: **seen** (participant count from the monitor) or **heard** (real audio above the noise floor).

> **THE BUG THIS FIXED**
>
> Counting participants means reading the meeting client's own interface, and when those selectors drift the bot decides it is alone in a room it can plainly hear. One interview was scored a no-show while the candidate sat in it for seven minutes. Hearing somebody speak cannot be argued with — a meeting client does not generate speech by itself. When heard-but-not-seen happens, the bot starts anyway and saves a diagnostic capture labelled `participants_miscounted`.

Two deadlines: a 3:00 interview waits until 3:05, then a final two minutes to 3:07. Entering the grace window is announced plainly, because that is the point at which a watching recruiter can still intervene. If somebody is visibly *knocking* at the deadline, the door is held open in 30-second extensions up to three minutes while admission keeps retrying — cancelling then would blame the wrong person. A candidate present but early gets one short greeting so they know the interviewer is there and not broken.

`cancelForNoShow()` marks the candidate `ABSENT` and records the run as **cancelled, not failed** — nothing broke, and a recruiter scanning a list should not have to open a red row to discover the candidate simply did not turn up.

The coding round

`startCodingRound()` first checks the editor actually loads (a 200 from `codingUrl`); if not, it says so, skips the round and tells the machine `{passed: 0, total: 0}` rather than waiting for a submission that cannot arrive. The link is posted to the meeting chat with two attempts (the control bar fades out, and the first click is often what brings it back rather than what opens the chat); on failure the interviewer says so aloud and a SYSTEM message tells the recruiter to send it manually. A timeout of `duration/3` minutes moves the interview on if no submission ever comes.

`presentCandidateCode()` is off by default and entirely best-effort. It opens the spectator page, **waits for the tab to carry the exact title Chromium is told to match**, and only then opens the picker.

> **WHY THE TITLE CHECK IS LOAD-BEARING**
>
> The picker is browser chrome, steered by a command-line title match rather than clicked. If nothing matches, Chromium does not give up — it shares whatever it finds first, which was usually the meeting itself. The call was shared back into the call and filled with feedback.

Speech, silence and teardown

`say()` chains onto `speechChain` so two turns can never talk over each other; a lost voice (`TTS_UNAVAILABLE` or `BROWSER_CRASHED`) is terminal, because there is no fallback channel to a candidate inside a call. `armSilenceTimer()` waits 45 s: in `GREETING` it defers to `greetingUnanswered()`, otherwise it prompts twice and then records "(no answer given)" and moves the script forward.

A browser crash or a meeting that ends is handled by **keeping the transcript**: if the interview had already started, the machine is told the candidate left, which marks it `INCOMPLETE` rather than discarding a real conversation as a failure. `finish()` is the single exit path — clear every timer, write the outcome, stop the monitor, stop presenting, leave the meeting politely, dispose audio and machine, close the browser, and emit `finished` so the manager releases the slot and the lock.

7.2 Browser, audio bridge and voice

> **backend/src/services/meetingBot/browser.ts** — 370 lines

Launches the Chromium the interviewer lives in. The bot signs in **once, by hand**, via `npm run bot:login`; every interview then reuses that user-data directory. Nothing here ever types a password.

Concern	Handling
Serverless detection	`serverlessHost()` checks for Vercel, AWS Lambda, Netlify and Azure Functions and throws `SERVERLESS_HOST` *before* the profile check — on such a host the missing profile is a symptom, and "run bot:login" is impossible advice. Cloud Run is deliberately not listed: it runs a normal long-lived process and works, provided it does not scale to zero.

Concern	Handling
Profile isolation	Chromium takes an exclusive lock on a user-data directory, so concurrent interviews need copies. When only one runs at a time the copy is pure cost — and worse, recent Chrome binds cookie encryption to the profile path, so a copied profile arrives *signed out*. The real profile is therefore used unless `MEET_BOT_MAX_CONCURRENT > 1`.
Fast cloning	A `DISPOSABLE` set skips caches, shader stores and Crashpad, and lock files are filtered out (copying them poisons the clone). A multi-hundred-megabyte copy becomes a few megabytes — it is on the critical path of every interview.
Chromium flags	`--use-fake-ui-for-media-stream` (no permission dialog nobody is there to answer), `--disable-blink-features=AutomationControlled` (Meet degrades its UI for what it thinks is an automated browser, and the controls the bot must click are part of what it takes away), autoplay policy, notification and popup suppression, memory flags for small instances, and both spellings of the auto-select-share-source flag.
Audio flags	`--mute-audio` on the injected path (remote audio still flows through WebRTC and Web Audio; only the speakers go quiet, avoiding an echo through the bot's own microphone) and `--use-fake-device-for-media-stream` so Meet sees an input device exists — its contents are never used, only its existence.
Channel fallback	Real Chrome is preferred because Meet treats it as a first-class client; if the configured channel is missing the launch falls back to Playwright's bundled Chromium with a warning rather than failing the interview.
Cookie injection	`GOOGLE_BOT_COOKIES` (base64 JSON) is supported for hosts with no persistent disk, though a real profile is preferred because it refreshes its own cookies and does not go stale.

`SHARE_TAB_TITLE = 'AI Interview Candidate Code'` lives here and is load-bearing: it must match the spectator page's `<title>` exactly and stay plain ASCII to survive the command line. `launchForLogin()` opens the master profile headful for the one interactive sign-in.

> **backend/src/services/meetingBot/injected/audioBridge.ts**
> 681 lines · ES5 string injected before the meeting's own bundle

> **WHY IT IS A PLAIN ES5 STRING**
>
> It runs before the meeting client's bundle, inside a page whose Content-Security-Policy will not load a module, and it must not depend on anything the page provides.

Output — the AI's voice into the meeting

`getUserMedia` is replaced with one returning a `MediaStreamAudioDestinationNode` the bridge owns. The meeting client treats that node as the microphone; the bot writes synthesised speech into it. No sound card, no virtual cable, no operating-system configuration. The track is **cloned per call** because the client stops the track it was given when the user toggles the microphone. A silent `ConstantSourceNode` runs forever, because Chrome lets a track producing nothing at all fall idle and the client then shows the bot as muted between questions. A video-only request is passed through untouched so the pre-join camera preview still behaves normally.

Input — three taps, tried in order

1. **RTCPeerConnection** — wrapped so every inbound audio track is tapped as it is added. This is how Google Meet and Teams work, and it is the cleanest: samples are read before they reach any speaker. The wrapper copies every

static off the native constructor (Meet reads `generateCertificate`, and a bare copy would call it with the wrong receiver).

2. **Web Audio** — `AudioNode.prototype.connect` is wrapped so anything routed to a speaker can also be routed to the bridge. Zoom's web client decodes audio in WebAssembly and plays it through its own `AudioContext`, so no remote track ever exists for tap 1 to find. Nodes cannot be connected across contexts, so each foreign context gets its own `MediaStreamDestinationNode` bridge.
3. **Media elements** — `<audio>` / `<video>` captured with `captureStream()`, which is non-destructive (unlike `createMediaElementSource`, which would re-route the element and silence the page's own playback).

> **WHY TAPS 2 AND 3 STAY DORMANT**
>
> Enabling them unconditionally would double-count on platforms where tap 1 already works — the same voice arriving twice, at twice the amplitude. They are switched on by Node only after twelve seconds of silence. But the *observation* hook is installed immediately: a client wires its audio graph once, when it joins audio, long before that timeout, so waiting to install the hook would leave nothing to see.

The capture chain

```
taps → inMixer → highpass 120 Hz (Q 0.5) → highpass 120 Hz → ScriptProcessor(4096)
                                                                        ↓
                                                          downsample, Int16, 250 ms frames
                                                                        ↓
                                                          base64 → window.__meetBotAudio → Node
```

Two cascaded high-pass stages, measured across the band: −28.9 dB at 50 Hz, −22.0 dB at 60 Hz, −3.4 dB at 100 Hz, and flat above 1 kHz. Mains hum, desk fans, air conditioning and traffic carry no words but are loud enough to hold voice detection open through pauses and drag the recogniser's gain down. A low shelf was tried and removed — it took as much voice as rumble. Downsampling **averages** the samples folding into each output sample rather than dropping every third one, which would alias sibilants into the speech band. Every node the bridge owns is marked `__meetBotInternal` so the Web Audio tap cannot feed the bridge back into itself.

The processor must reach a destination to run at all, so its output goes through a zero-gain sink. `status()` reports context state, sample rate, mic injection, track counts, which tap found audio, frames sent, peak sample and the last five errors — between them enough to distinguish "deaf", "connected to silence" and "working" from outside the browser.

Playback

`speak(base64, rate)` decodes PCM into an `AudioBuffer` and schedules it against a **playhead** rather than playing on arrival, so a sentence streamed as twenty small pieces is heard as one continuous utterance. `remaining()` returns the seconds still queued; `stopSpeaking()` cuts the current utterance short. `speakWebSpeech()` is the alternative path, with a guard timeout because Chrome silently drops long utterances after about fifteen seconds and fires no event at all.

> **backend/src/services/meetingBot/audioManager.ts** 449 lines

Owns both directions and, crucially, **keeps them apart**. The one place they meet is the listening gate: while the interviewer is speaking, transcripts are dropped rather than treated as an answer — the candidate's microphone will pick the AI's voice out of their speakers, and without the gate the interview would answer itself. A 600 ms `LISTEN_RESUME_DELAY_MS` covers the tail of the speakers and the room's reverb.

WHY CAPTURE IS 48 KHZ, NOT 16 KHZ

16 kHz was chosen for being Deepgram's native speech rate and a third of the bytes. The bytes were true; the quality was not. Getting from the browser's 48 kHz to 16 kHz meant averaging every three samples, and a three-tap box filter is a poor anti-alias filter — around 4 dB down at the new Nyquist, so everything above 8 kHz folded back into the speech band as noise. Sibilants suffered most, which is exactly what makes a transcript full of near-miss words. Deepgram resamples internally with a filter built for the job, so handing it the original 48 kHz is both simpler and better, at a cost of about 170 MB of bandwidth for a half-hour interview and nothing else.

`hasHeardSomeoneSpeak` scans frames for any sample above ~1 % of full scale — the most dependable evidence that somebody else is present. Two watchdogs: at 12 s with no audio the fallback taps are enabled; at 40 s a `deaf` event is emitted with the bridge's full diagnostic state. Deafness is not fatal on its own (the candidate may simply be muted) but it is surfaced to the recruiter, because the alternative is a transcript full of "(no answer given)" with no explanation.

backend/src/services/meetingBot/tts.ts 456 lines

Driver	Behaviour
deepgram (default)	Synthesises server-side and returns raw PCM, which goes straight into the synthetic microphone. Nothing to set up on the host, works headless, and reuses the key the bot already needs for transcription. Audio is **streamed** — forwarded as it arrives so the candidate hears the first words while the rest is still being generated; a whole-response wait added a second of dead air to every question. Only whole 16-bit samples may cross into the page, or a split sample is reassembled as noise.
webspeech	The browser's SpeechSynthesis API. Free and keyless, but it plays to a sound device and hands back no samples, so on its own it reaches nobody — it works only when the host routes the output device into the browser's microphone with a virtual audio cable.

Non-English voices. Deepgram Aura is English-only, so twelve locales are routed to Microsoft Edge neural voices (`hi-IN-SwaraNeural`, `te-IN-ShrutiNeural`, `ta-IN-PallaviNeural`, `bn-IN-TanishaaNeural`, `mr-IN-AarohiNeural`, plus Spanish, French, German, Portuguese, Japanese, Mandarin and Arabic). Edge returns MP3, so each chunk is decoded and pushed as it arrives, with the decoder keeping state across chunk boundaries; a promise chain keeps chunks strictly in arrival order, since two racing would interleave their samples.

SPEAKABLE() — WHERE THE "SYNTHETIC" SOUND ACTUALLY CAME FROM

Not audio quality. A voice model reads what it is given, and "I am going to be taking your interview, and I will leave time at the end" is stilted no matter how good the synthesis. This function applies contractions (ordered longest-first, case-preserving), turns em dashes and ellipses into commas (an em dash has no spoken equivalent and Aura renders one as a hard stop mid-clause), and normalises curly quotes. It is applied at the last possible moment, so the transcript the recruiter reads keeps its proper punctuation and only the audio is colloquial. `have` is deliberately excluded: English contracts it only as an auxiliary, and a blanket rule produced "we've about fifteen minutes" and "do you've any questions".

`waitUntilSpoken()` polls `remaining()` until the audio has actually left the graph — "spoken" must mean heard, not submitted, or the interviewer asks its next question over the top of the previous one.

7.3 Platform drivers, selectors and diagnostics

backend/src/services/meetingBot/platforms/types.ts 161 lines

The `PlatformDriver` interface every platform implements: `matches`, `parse`, `join`, `observe`, `admitWaiting`, `sendChat`, `presentTab`, `stopPresenting`, `leave`, plus `requiresSignIn`. The shape of the problem is identical everywhere — open a link, camera off, microphone on, ask to join, wait to be admitted, watch who else is in the room — so everything above this line (the audio bridge, the interview engine, the scheduler, the dashboard) is written once and works for all three.

`MeetingObservation` distinguishes `participants: 0` ("could not be read") from a confident count of one, and keeps `waitingRoomOccupied` separate from the head count — a candidate visibly waiting to be admitted has not failed to turn up, they have failed to be admitted, and telling the recruiter the wrong one wastes their time chasing the wrong person. `assertNotAborted()` is called between every join step because a join can sit in a lobby for ten minutes. `waitUntilDue()` holds the pre-join screen in one-second steps until the scheduled moment.

backend/src/services/meetingBot/selectors.ts 426 lines

The selector engine every driver is built on. All three clients ship UI changes continuously and all three generate their CSS class names, so each control is described as an **ordered list of strategies** — accessible role and name first, then stable data attributes, then aria-label substrings, then visible text — and the first that resolves to a *visible* element wins. Searching covers every frame, not just the top one, because Teams renders its whole pre-join in an iframe.

Helper	Purpose
`findFirst / isPresent`	Polling search returning null rather than throwing — most groups are optional, and callers that require an element raise their own `BotError`.
`setToggle`	Drives a device toggle to a known state. Never clicks blind: an unread toggle is as likely to turn the camera *on* as off. Each platform supplies its own state reader.
`CAMERA_TRY`	One attempt, 2.5 s. Whether the bot's camera is on changes nothing — the server has no webcam. The default ten-second budget was ten seconds of being late, spent finding a control that does not exist.
`wakeControls`	Moves the mouse across the viewport. All three clients fade their controls out after a few seconds without pointer movement, and a bot's pointer never moves — so chat and share buttons sit in the DOM but invisible, which is exactly what `findFirst` will not return.
`admitAllWaiting`	Clicks every visible Admit control, up to five times, because each admission re-renders the list. **Deny is never in any selector group** — a bot that could refuse someone entry will eventually refuse the wrong person.
`startPresenting`	Opens the share flow and judges success by the stop-sharing control appearing, because the menus happily accept clicks that lead nowhere.
`readAriaMuted`	Reads `data-is-muted`, else infers from the accessible name ("Turn off microphone" only says that while it is on), else `aria-pressed` / `aria-checked`, else null.

platforms/googleMeet.ts 722 lines · requiresSignIn: true

Meet is the strictest about who may join — anonymous participants are refused by many meetings — so this driver expects a signed-in profile, while still handling the guest name field a guest-enabled meeting shows.

- **parse** validates the three-four-three meeting code and canonicalises to `https://meet.google.com/<code>`, dropping tracking and account-hint query strings.
- `isInCall()` **requires two conditions**: a leave control present *and* a join control absent. `[data-meeting-code]` used to count as proof and is present on the pre-join screen too, which made the bot skip pressing "Join now" entirely and then sit on "Ready to join?" for the whole interview reporting itself as joined.
- **The guest name box is checked, not attempted.** "Ask to join" stays disabled while it is empty, so failing loudly here is much kinder than clicking a dead button for forty seconds and reporting that no join control was found. Three fill attempts, falling back to real key events.
- `pressJoin()` **clicks up to four times.** Meet re-renders the pre-join screen when a device changes state — a machine with no webcam does this on its own — and the click lands on an element replaced a moment later. A *disabled* join button is reported as its real cause (usually an empty guest name) rather than as a missing control.
- **Admission refused is disambiguated.** Meet turns away an uninvited stranger and a host-refused guest with the same screen. What tells them apart is whether this bot ever proved who it was: a page that was shown the guest-name box is recorded in a `WeakSet`, and its refusal is reported as `SIGN_IN_REQUIRED`, not `ADMISSION_DENIED`. Reporting the wrong one sends the recruiter to argue with a host who did nothing.
- **admitWaiting** tries the notification card, then wakes the controls, then opens the People panel.

> **platforms/zoom.ts** — 599 lines · requiresSignIn: false

The link a recruiter copies opens a page whose whole job is to launch the desktop app, so the driver rewrites `/j/<id>` to the web client `/wc/<id>/join` and goes straight there — the same destination a person reaches, minus the page selling them the native app. The encrypted `pwd` passcode is preserved, or the bot is stopped by a prompt it cannot answer. The recruiter and candidate keep the link they know; only the bot uses the web client. Zoom's pre-join offers only a camera toggle, so the microphone is set after joining, and `joinComputerAudio()` is essential — without it the bot is in the meeting with no audio device at all, which looks like a working join and sounds like total silence. Leaving requires confirming a second dialog.

> **platforms/teams.ts** — 583 lines · requiresSignIn: false

Takes the launcher's "Continue on this browser" option, then the guest path — which is the candidate's path too. Teams renders pre-join inside an **iframe** and is the slowest of the three to get there; the name box is the landmark that says the screen is ready, or the join button when a signed-in account skips it. The query string is load-bearing in a way it is not for Meet or Zoom: the `context` parameter carries the tenant and organiser, so the URL is kept whole and unmodified. Anonymous join is a tenant setting; when it is off, Teams demands a sign-in and the bot reports that rather than trying to get around it.

> **meetingMonitor.ts** — 197 lines

Polls the driver every 3 seconds and debounces in one place for all three platforms. A state change must hold for **three consecutive readings** before it is believed, because all three clients re-render their participant lists during a layout change and briefly report one participant, and all three can flash an "ended"-looking screen mid-reconnect. Emits `candidateArrived`, `alone`, `ended`, `admitted`, `admissionStuck` and `snapshot`. `admitWaiting` is called on *every* tick whether or not anyone appears to be waiting — the recruiter usually creates the meeting from the same account the bot signs in as, which makes the bot the host, and a host that never presses Admit leaves the candidate in the waiting room for the entire interview. A throw from `admitWaiting` is logged, not swallowed: swallowing it made a broken selector look exactly like an empty waiting room.

errors.ts 183 lines

A closed set of 32 `BotErrorCode`s, each with recruiter-facing guidance and a `retryable` flag. Nine codes are retryable (browser launch, crash, page timeout, pre-join not found, join control not found, meeting not started, audio bridge, network, capacity); a bad link or a locked-out account never is. `toBotError()` normalises Playwright's prose — closed browser, `net::err_*`, timeouts, missing executable — into codes, once, instead of at every call site. The file also owns `MeetingPlatform`, `PLATFORM_LABEL` and `ParsedMeetingLink`, whose `displayUrl` / `joinUrl` split is what lets Zoom show one link and drive another.

debugCapture.ts 131 lines

Saves what the page actually looked like when a join failed: a screenshot plus a text dump of the visible text **and every clickable control's accessible name and data hooks** — the second half is what a selector is written against, so it is the more useful one. A broken selector looks exactly like a broken network from outside (`PRE_JOIN_NOT_FOUND` and nothing else), and these are the only way to fix a selector for a meeting you cannot reproduce. Captures older than three days are pruned; a failure to record a failure never replaces it.

joinMeeting.ts · platforms/index.ts

`joinMeeting()` is the thin dispatcher that resolves the driver, applies configured defaults and reports whether admission was required and how long the join took. `platforms/index.ts` holds the driver registry, `detectPlatform()`, `parseMeetingLink()` (called before a run is stored *and* before the browser launches) and `isSupportedMeetingLink()`. Adding a fourth platform means writing one driver and adding it to the array.

8 · Backend — scripts

> Eighteen scripts under `backend/scripts/`, all run through `tsx`. They fall into three groups: end-to-end verification against a live API, unit-style checks of pure logic, and operations tooling for the meeting bot.

Command	File	What it proves or does
`npm run e2e`	`e2e.ts`	The whole product: creates a session, waits for question generation, invites a candidate, runs the entire interview over Socket.IO with realistic answers and timings, then verifies the report, exports, ranking and analytics. Minutes long, because every conversational turn is a real model call.
`verify:coding`	`codingtest.ts`	Optimal, brute-force and broken submissions score correctly; hidden cases run and are graded.
`verify:codingflow`	`codingflow.ts`	A coding question is generated, saved, emitted by the state machine, dry-run, and graded on the real compiler.
`verify:resume`	`resumetest.ts`	PDF extraction, profile analysis, missing-skill detection and tailored question generation.
`verify:resumedb`	`resumedbtest.ts`	The resume file is stored in Postgres and served back byte-identical.
`verify:noshow`	`noshowtest.ts`	Nudge inside the grace window, absent after it, future sessions untouched, idempotent. Two sessions are backdated so both stages fire in one tick.
`verify:gate`	`gatetest.ts`	The room does not open early, unused links expire, and a reconnect still works. Pure logic, no API needed.
`verify:ratelimit`	`ratelimittest.ts`	Quota hints are parsed in h/m/s form from Groq's actual message, and the global cooldown engages and extends.
`verify:incomplete`	`incompletetest.ts`	Cutting an interview short marks it `INCOMPLETE` and produces no score, while a finished one is `COMPLETED` and scored.
`verify:speech`	`speechtest.ts`	The Deepgram key is accepted, a live socket opens, and transcripts come back.
`diagnose`	`diagnose.ts`	Recent interviews: what was captured (turns, submissions, frames, insights), what is missing, and why.
`bot:login`	`botlogin.ts`	The one-time interactive sign-in. A window opens, a human signs in, and the session is left in the profile at `GOOGLE_BOT_PROFILE_PATH`. All three platforms share one profile.
`bot:login:vnc`	`botlogin-vnc.sh`	The same sign-in inside the container, viewable over VNC — necessary because a Windows profile encrypts its cookie store with DPAPI bound to that machine's user account, so it arrives on a Linux server unreadable.
`bot:export-cookies`	`export-cookies.ts`	Exports the profile's cookies as base64 for hosts with no persistent disk.

Command	File	What it proves or does
`bot:test`	`bottest.ts`	Proves the whole join-and-speak path against a real meeting, with no database, scheduler or interview involved. Fails in the same places a real interview would.
`seed`	`prisma/seed.ts`	A demo recruiter (`recruiter@demo.com` / `demo1234`) with sessions, candidates and a finished interview, so a fresh database renders something real.
—	`makeresume.ts`	Generates the fixture PDF the resume tests read.

9 · Frontend

> Next.js 14 App Router, React 18, TypeScript, Tailwind, Recharts, Monaco, socket.io-client and axios. Roughly 10,300 lines across 44 files. Every page is a client component; all data comes from the API over axios.

9.1 App shell and routing

Route group	Purpose
`app/layout.tsx`	Root. Loads Inter and JetBrains Mono as CSS variables, sets metadata and a viewport that keeps pinch-zoom working (disabling it is an accessibility failure), and wraps everything in `AuthProvider` and `ToastProvider`.
`app/page.tsx`	The marketing landing page: floating navbar, hero with an animated visual, six capability cards and a four-step "how it works" section.
`app/(auth)/`	Login and register, with a shared centred layout. Both redirect to the dashboard when already signed in.
`app/(app)/layout.tsx`	The authenticated shell. Waits for the refresh attempt to settle, redirects to `/` when there is no user, then renders the sidebar plus a max-width content column.
`app/interview/[token]/`	Candidate-facing and unauthenticated — the room, the coding editor and the read-only code mirror. Deliberately outside the app shell: no sidebar, no login.

9.2 Recruiter pages

Page	Contents
`/dashboard`	Stat tiles (sessions, candidates, completed, absent, reports), completion / no-show / hire rates, average scores, and an upcoming-interviews list with relative times and copyable join links.
`/ai-interviews`	The meeting-bot list: status chip per interview, live indicator, quick filters and search, and the queue state (enabled, running, capacity, next launches).
`/ai-interviews/new`	The one-screen booking form, plus the bulk-import flow: upload → preview with per-cell errors → fix in place → book. 588 lines.
`/ai-interviews/[id]`	The live console. A six-stage progress track (Scheduled → Starting → Joining → Joined → Interviewing → Completed), a running clock, and four tabs — **Live** (streamed transcript with interim text), **Questions**, **Transcript**, **Details**. Backed by `useMeetBot` over the socket with a 6-second poll as a fallback so status still advances if the websocket cannot connect. 773 lines.
`/sessions`	Paginated list with search, status / type / date filters, candidate and question counts.
`/sessions/[id]`	Four tabs — Candidates, Questions, Results, Settings — plus the meeting link and an Excel export. Polls briefly after load because questions generate in the background.
`/sessions/[id]/compare`	Side-by-side comparison of 2–6 candidates with score bars per dimension.
`/candidates`	The cross-session directory with search, pagination and per-candidate session history.

Page	Contents
`/reports`	Recent reports grouped by role, searchable, with score and recommendation chips.
`/reports/[id]`	The full report: five tabs — Overview, Skills, Coding, Transcript, Signals — with score bars, evidence-cited strengths and weaknesses, red flags, PDF/Excel download, feedback email and recommendation override. 735 lines.
`/analytics`	Stat tiles, a 14-day invited/completed time series, a recommendation distribution ordered strongest-first, strongest and weakest skills, and a per-session comparison table.
`/integrations`	ATS integrations: add (provider, name, webhook URL, optional key), test against the most recent report, view the last 50 sync logs, delete.

9.3 Candidate pages

> **`app/interview/[token]/page.tsx`** 223 lines

Fetches the interview context and branches on the gate verdict. `TOO_EARLY` renders a **live countdown** that lets the candidate in automatically the moment it reaches zero; `EXPIRED`, `ALREADY_COMPLETED`, `MARKED_ABSENT` and `CANCELLED` each render their own explanation. When open, it shows `PreCheckScreen`, then hands the chosen devices to `LiveRoom`.

> **`app/interview/[token]/code/page.tsx`** 194 lines

The coding editor for an interview running inside a meeting call. Authenticated by the same access token, so there is no login and nothing new to remember. It joins the `/coding` namespace and broadcasts every keystroke batch so the mirror stays current, and it shows whether the mirror is connected. The candidate keeps the meeting open in another window and talks through their thinking as they work; **the submission itself is what tells the interviewer to continue**.

> **`app/interview/[token]/code/live/page.tsx`** 148 lines

The read-only view the interviewer presents into the meeting. Everything about it is shaped by being seen as a shared screen on someone else's monitor: large type, high contrast, dark background, no chrome, nothing interactive. Its `document.title` is set to the fixed ASCII string `AI Interview Candidate Code`, which must stay in step with `SHARE_TAB_TITLE` in the backend's `browser.ts` — the bot passes it to Chromium as a flag to auto-select this tab for sharing.

9.4 Components

UI kit — `components/ui/index.tsx` (484 lines)

One file, no component library. `Button` (five variants × three sizes, with a loading state), `Card` family, `Input`, `Textarea`, `Select`, `Field` (label, hint, error), `Checkbox`, `Badge` with a `STATUS_TONE` map covering every status enum in the system, `StatusBadge`, `humanise()`, `Spinner`, `Skeleton`, `EmptyState`, `Alert`, `scoreTone()`, `ScoreBar`, a generic typed `Tabs<T>` and a focus-trapping `Modal`.

Charts — `components/charts/index.tsx` (332 lines)

A small Recharts wrapper with a shared `VIZ` token set so every chart reads as one system: `ChartFrame` (title, description, empty state, responsive container), `BarList`, `TimeSeries` and `StatTile`.

Interview components

Component	Behaviour
`PreCheckScreen` 335 lines	Microphone and camera check with a live level meter and preview, device pickers, a warning when the browser has no speech recognition, the optional resume upload, and a Ready gate that hands back the chosen `DeviceChoice`.
`LiveRoom` 682 lines	The room itself. Owns the Socket.IO connection, the phase machine (`connecting → greeting → speaking → listening → thinking → coding → ended`), the transcript strip, the round label, the timer, the typed-answer fallback, the doubt box, camera toggle and the coding panel. Composes `useSpeaker`, `useListener`, `useAudioStreamer`, `useVideoAnalysis` and `useProctoring`.
`VoiceOrb`	The two-sided presence indicator: which orb is lit tells the candidate whose turn it is without them having to read anything. The ring scales with the audio level.
`CodingPanel` 377 lines	Monaco (dynamically imported, SSR off, because it is heavy and browser-only), language picker, problem statement, constraints, sample tests, Run (dry) and Submit (graded), per-case results and a hint button.
`ResumeUpload`	Drag-and-drop upload showing extracted characters, pages, detected skills and how many tailored questions were written — and saying plainly when the file is stored but analysis is still pending.

Session and interview management components

- **CandidatesTab** (420) — add one, bulk upload, copy join link, resend invite, remove, per-row status, report link and `EvaluationStatus`.
- **QuestionsTab** (364) — the question list with inline add / edit / delete, disabled once the interview has started.
- **ResultsTab** (289) — the leaderboard, shortlist and comparison entry point.
- **SettingsTab** (237) — editable session configuration.
- **ResumePanel** (293) — the parsed profile: headline, years, skills, roles with highlights, projects, education, missing JD skills, claims to probe and gaps.
- **EvaluationStatus** (106) — turns the seven evaluation states into one honest line, with a *Retry now* button where retrying makes sense.
- **CandidateImport** (286) — the editable preview table for bulk import, highlighting the exact cells that are wrong.
- **QuestionEditor** (413) — the full question editor including the coding payload: constraints, starter code, optimal complexity and the visible/hidden test-case grid.
- **Sidebar** / **PageHeader** / **Toast** — navigation with a mobile drawer, page titles with actions, and a context-based toast queue.

9.5 Hooks and libraries

Module	Behaviour
`lib/api.ts`	The axios client. **The access token lives in memory only**; the refresh token is an httpOnly cookie, so a page reload silently re-authenticates rather than leaving a long-lived token in localStorage. A single-flight refresh means concurrent 401s share one call instead of stampeding the endpoint, and a failed refresh redirects home. `errorMessage()` unwraps the server's own phrasing — including the 429 quota message with its concrete wait — and distinguishes a timeout from an unreachable backend. `downloadFile()` handles blob downloads.
`lib/utils.ts`	`cn()` (clsx + tailwind-merge), date/time/duration/clock formatters, `relativeTime()` falling back to an absolute date beyond a month, `toDateTimeLocal()`, `initials()`, a deterministic `avatarColor()` and `truncate()`.
`lib/meetInterview.ts`	The vocabulary of a meeting interview: the 15 bot statuses surfaced verbatim, the interview and evaluation types, and helpers `statusMeta`, `canStart`, `canStop`, `canRetryReport`, `evaluationLabel`, `evaluationTone`. Recruiters need to tell "nobody has let the bot in yet" apart from "the bot is in and waiting" — collapsing those into one "connecting" state would hide the only two things they can act on.
`hooks/useSpeech.ts`	`useSpeaker()` wraps SpeechSynthesis with voice selection per language and a promise that resolves when the utterance ends; `useListener()` wraps the Web Speech recognition API with interim results, restart-on-end and error classification. `speechSupported()` is what drives the typed-answer fallback on Firefox and Safari.
`hooks/useAudioStreamer.ts`	Streams microphone audio to our own server, which relays it to Deepgram. **The recorder runs continuously for the whole interview and is never restarted between turns** — MediaRecorder emits a WebM container header on its first chunk only, so restarting mid-session pushes a second header into a stream Deepgram is already parsing, after which it stops returning transcripts entirely. Whether speech counts as an answer is decided by the server, via `listening_started` / `listening_stopped`, not by muting the microphone.
`hooks/useVideoAnalysis.ts`	Samples the webcam to a 64×48 canvas, detects skin-toned pixels with the Kovac RGB rule, and derives face presence, inter-frame motion and gaze steadiness. **No pixel data ever leaves the device** — only the four aggregate numbers, batched, and only when the recruiter enabled video analysis.
`hooks/useProctoring.ts`	Notices tab switches, warns clearly and reports them with the duration away; blocks copy, paste and the context menu during the interview. The file states its own honesty constraint: a browser cannot actually prevent tab switching, and pretending otherwise would be security theatre.
`hooks/useMeetBot.ts`	The recruiter's live subscription. The socket carries what is happening now; the transcript endpoint carries what already happened, and `mergeTranscript()` joins them without duplicates so a recruiter opening the page twenty minutes in sees the full history.

10 · Cross-cutting behaviour

10.1 The scoring model, end to end

```
MEASURED (no model involved)
  TranscriptTurn.latencyMs / durationMs / confidence / wordsPerMinute
        │
        ├─ extractSignals()      17 measurements
        └─ scoreCommunication()  fluency, confidence, clarity,
                                 grammar, vocabulary, pace  → overall

  RealtimeInsight rows  — summarise() → composure 0..10
  VideoAnalysisFrame    — summarise() → confidenceScore 0..10
  CodingSubmission      — real execution → correctness

JUDGED (one smart-model call, constrained)
  technical{knowledge, problemSolving, logicalThinking,
            projectUnderstanding, domainExpertise}
  behavioral{leadership, teamwork, adaptability,
             ownership, learningMindset}
  skillBreakdown[], strengths[], weaknesses[], improvements[],
  redFlags[], summary, recruiterFeedback, candidateFeedback

COMBINED
  communicationScore = measured × 0.8 + composure × 0.2
  codingScore        = mean( correctness × 0.6 + quality × 0.4 )
  overall            = Σ(dimension × weight) / Σ(measurable weight)

GATED
  ≥3 red flags                          → REJECT
  overall < pass−2 / < pass             → REJECT / CONSIDER
  ≥8.5 & tech ≥8 & comm ≥7 & no flags   → STRONG_HIRE
  ≥pass+1 & tech ≥pass                  → HIRE
  otherwise                             → CONSIDER
```

Three properties are worth stating plainly:

- **Communication is not an LLM opinion.** It is computed from measured speech and is reproducible.
- **Unmeasured dimensions are not zeros.** Their weight is redistributed across what was actually observed.
- **The recommendation is not just the number.** Hard gates sit on top, so a fluent talker with no technical substance cannot reach Strong Hire and three red flags force a Reject regardless of score.

10.2 Failure handling

Failure	Response
LLM quota exhausted	The provider's own "try again in 1h5m" is parsed; a per-provider global cooldown pauses further calls; callers get **429** with `Retry-After` and a message naming the wait; question generation falls back to the deterministic template set so a session is never left unusable.
Report generation fails	`EvaluationQueue` retries one at a time with exponential backoff to a 30-minute cap and 5 attempts. A permanent failure (nothing was said) stops immediately. The recruiter sees *Generating report…*, *Report retrying in 8 minutes* or *Report failed* with a **Retry now** button.

Failure	Response
Model returns bad JSON	The validation error is fed back to the model as a repair instruction and the call retries twice before surfacing.
Resume analysis fails	The file and its extracted text are already committed; `resumeParsedAt` stays null and the scheduler finishes the job later.
Candidate disconnects	45-second reconnect grace in the browser room. If they never return, the interview is `INCOMPLETE` — the transcript is kept and deliberately not scored.
Bot browser crashes mid-interview	If the interview had started, the machine is told the candidate left, preserving the transcript, rather than discarding a real conversation as a failure.
Server restarts during a run	`recoverOrphans()` releases stale locks at boot and marks those runs `FAILED / BROWSER_CRASHED` with a plain explanation, so they are visible instead of stuck.
Meeting selectors drift	Ordered multi-strategy selectors, all-frames search, control-waking, streak debouncing, and a screenshot plus accessible-name dump saved on the attempt that gives up.
Bot is deaf	Fallback audio taps at 12 s; an explicit `deaf` event with full bridge diagnostics at 40 s, surfaced to the recruiter rather than producing a silent transcript.
Deepgram unavailable	The browser room falls back to Web Speech recognition and says so via `speech_unavailable`. The meeting bot cannot — it has no browser to fall back to — so it fails loudly rather than interviewing into the void.
Coding editor not deployed	Probed at booking time and again before the round starts; the round is skipped with an explanation rather than handing the candidate a 404 mid-interview.
Email silently rejected	Verified-sender check at boot; sends refused up front; status reported by `/health`.
Session wedges	Lifetime guard on every bot session, hard stop at 1.5× duration in the state machine, per-case execution timeouts, and 90-second abort signals on model calls.

10.3 Security posture

Area	What is true
Recruiter auth	bcrypt at cost 10, a 30-minute access token held in memory, a 7-day httpOnly refresh cookie, single-flight refresh, and a constant-time-ish login that compares against a dummy hash for unknown addresses.
Candidate auth	An unguessable UUID in the URL granting access to exactly one interview, expiring implicitly through the join gate when the interview is complete, absent or past its window.
Authorisation	Every recruiter query is scoped `interviewSession: { userId }`. Ownership is proven through the relation, never inferred from an id in the URL.
Secrets	Deepgram and LLM keys stay server-side; audio is proxied through this server precisely so a browser-held key cannot be lifted from the network tab. ATS API keys are never returned by any read path — only `hasApiKey`.
Hidden test cases	Filtered out of every response the candidate can reach — the socket payload, the meeting coding endpoint, and the graded result, which returns only aggregate hidden pass counts.

Area	What is true
Video privacy	Frames are sampled to a 64×48 canvas and reduced locally. No frame, image or face embedding leaves the candidate's machine, and nothing is stored server-side beyond the four aggregates.
What the AI cannot do	Structurally, not by asking politely: it cannot choose the next question, reveal expected answers or scores, skip ahead, or end the interview early. `sanitise()` overrides an unprompted `END_EARLY`, replaces an explanation masquerading as a clarification, and truncates an acknowledgement that drifts into a second introduction.
Bot boundaries	It never creates a meeting, never types a password, never answers a security challenge, and never clicks Deny — Deny is not in any selector group, because a bot that could refuse someone entry will eventually refuse the wrong person.

STATED LIMITATION — CODE EXECUTION IS SANDBOXED, NOT ISOLATED

Submitted code runs as a child process with a blank environment, a temp working directory, a wall-clock limit, an output cap, memory caps and a static ban on process/network/filesystem APIs. That stops accidents and casual abuse. It is
NOT
a hardened boundary: a determined attacker who can submit code can still read files the server user can read. For untrusted candidates, run the backend in a container with no network egress and a read-only filesystem, or set `CODE_EXEC_ENABLED=false`.

OTHER HONEST LIMITATIONS

Interview state is held
IN MEMORY
, so a backend restart mid-interview loses the in-flight session and all sockets for one interview must reach the same instance (sticky sessions or the Socket.IO Redis adapter behind a load balancer). Emotion detection is heuristic, derived from skin-tone centroid stability and motion, not a trained model. Browser speech recognition is Chrome and Edge only; other browsers fall back to typed answers. Question quality depends on the LLM — the deterministic fallback keeps a session usable but its questions are generic rather than JD-specific.

11 · Configuration & deployment

11.1 Required environment

Variable	Notes
`DATABASE_URL`	PostgreSQL connection string
`JWT_SECRET`	≥ 10 characters
`JWT_REFRESH_SECRET`	≥ 10 characters, different from the above
`GROQ_API_KEY` or `MISTRAL_API_KEY`	At least one; naming `LLM_PROVIDER` without its key fails at boot
`DEEPGRAM_API_KEY`	Optional for the browser room (it falls back to Web Speech), **required** for the meeting bot, which has no browser to fall back to
`NEXT_PUBLIC_API_URL`	The frontend's only variable, in `frontend-v2/.env.local`

11.2 Key optional settings

Variable	Default	Purpose
`LLM_PROVIDER`	auto	`auto` splits fast/smart across Groq and Mistral when both keys exist
`GROQ_FAST_MODEL / _SMART_MODEL`	8b / 70b	Conversation vs. generation and scoring
`APP_URL / API_URL`	—	Where candidate links point; mismatches are warned about at boot
`API_KEY_FOR_EMAIL / EMAIL_FROM`	—	Brevo. Without a key, email is logged. `EMAIL_FROM` must be a verified sender
`SCHEDULER_ENABLED`	true	**Set false on every replica but one**, or candidates get duplicate reminders
`NO_SHOW_GRACE_MINUTES`	5	Drives the nudge, the link expiry and the absent marking together, so they can never disagree
`CODE_EXEC_ENABLED / _TIMEOUT_MS`	true / 5000	Code execution kill switch and per-case wall clock
`MEET_BOT_ENABLED`	true	Master switch for the meeting interviewer
`MEET_BOT_JOIN_LEAD_MINUTES`	0	Zero means a 3:00 interview is joined at 3:00
`MEET_BOT_WARMUP_SECONDS`	75	How early the browser starts; the join press is still held to the scheduled second
`MEET_BOT_CANDIDATE_WAIT / _GRACE_MINUTES`	5 / 2	The two-phase wait before cancelling for a no-show
`MEET_BOT_MAX_CONCURRENT`	2	Each is a real browser; 1 on a 2 GB instance
`MEET_BOT_TTS / _TTS_MODEL`	deepgram	`webspeech` needs a virtual audio cable on the host
`MEET_BOT_HEADLESS`	false	False in containers too — the Dockerfile runs a headed browser under Xvfb

Variable	Default	Purpose
`MEET_BOT_SHARE_CODE_SCREEN`	false	Off by default; a picker miss shares the call into itself
`GOOGLE_BOT_PROFILE_PATH`	./.meet-bot-profile	Must be on a persistent volume, or the bot is signed out on every deploy
`GOOGLE_/ZOOM_/MS_ credentials`	—	Only needed to *create* meetings through provider APIs

11.3 Running locally

```
npm run setup                # installs backend + frontend dependencies
cp backend/.env.example backend/.env      # fill DATABASE_URL and an LLM key
npm run db:push              # create the schema
npm run seed                 # optional: recruiter@demo.com / demo1234
npm run dev                  # API on :5000, web on :3000

npm run typecheck            # both apps
npm run build                # both apps
npm run e2e                  # full interview against a running API
```

Use Chrome or Edge for the built-in room: speech recognition relies on the Web Speech API, and other browsers fall back to typed answers automatically.

11.4 Deployment

> **backend/Dockerfile**

Built on `mcr.microsoft.com/playwright:v1.62.1-noble` — the tag **must** match the Playwright version in `package.json`, because Playwright looks for browsers in a version-stamped directory. One apt layer installs three things for three separate reasons: **x11vnc** for the one-time Google sign-in inside the container (a Windows profile encrypts its cookie store with DPAPI bound to that machine's account, so it arrives on Linux unreadable), and **g++** plus **default-jdk-headless** so a candidate who spends twenty minutes on a C++ or Java solution does not discover at submit time that it cannot run.

Dev dependencies are deliberately kept after the build: the generated Prisma client lives in `node_modules`, and `npm prune` can take it with it. The entrypoint is `xvfb-run … node dist/server.js` — a headed browser on a machine with no monitor, because meeting clients are measurably more cooperative that way.

> **backend/fly.toml · render.yaml · .github/workflows/fly-deploy.yml**

- **Fly.io** — 2 shared CPUs, 2 GB RAM (256 MB cannot start Chromium at all), `swap_size_mb = 1024` declared *above every table header* (a bare key after `[[vm]]` becomes part of that table and is silently ignored — which left the machine with no swap and the C++ compiler being OOM-killed during meetings), a mounted volume for the bot profile, `TZ` set explicitly (a container defaults to UTC and a 3:00 pm IST interview is otherwise emailed as 9:30 AM), and `auto_stop_machines = 'off'` with `min_machines_running = 1`.
- **Render** — the same Docker image, `numInstances: 1` because the state machine is in memory and the scheduler must run in exactly one place; JWT secrets are generated, everything else is set in the dashboard.
- **GitHub Actions** — deploys the backend to Fly on push to `main`, but only when `backend/**` changed: a frontend-only push would otherwise rebuild a 2 GB image and restart the machine, which if it lands during an interview kills a

live meeting for no reason. Concurrency is limited to one deploy at a time.

- **Vercel** — the frontend, with `NEXT_PUBLIC_API_URL` pointing at the public API origin.

> **THE SINGLE MOST IMPORTANT DEPLOYMENT FACT**
>
> The meeting bot cannot run on a serverless host. It holds a browser open for the whole interview, needs a signed-in Chromium profile that survives between runs, and needs an always-awake scheduler. Platforms that scale to zero — or throttle CPU between requests, as Cloud Run does by default — will freeze the bot mid-interview. The code detects Vercel, Lambda, Netlify and Azure Functions and refuses with an explanation rather than failing obscurely.

Production checklist

- Replace the development JWT secrets with random values; rotate any keys committed during development.
- Serve both apps over HTTPS — `getUserMedia` and the Web Speech API require a secure context.
- Restrict CORS in `server.ts` from `origin: true` to the actual frontend origin.
- Run code execution in a container with no network egress and a read-only root filesystem, or disable it.
- Set `SCHEDULER_ENABLED=false` on all but one replica; enable sticky sessions or the Redis adapter if you run more than one.
- Use `prisma migrate deploy` rather than `db push`.
- Verify `EMAIL_FROM` in Brevo and confirm `/health` reports `emailSender: verified`.

12 · Appendix — API surface

12.1 REST endpoints

Public — no authentication

Method	Path	Purpose
GET	`/health`	Database, email sender, speech, LLM provider, active interviews, meeting-bot state, uptime
POST	`/api/auth/register`	Create an account; sets the refresh cookie
POST	`/api/auth/login`	Sign in
POST	`/api/auth/refresh`	Rotate tokens from the httpOnly cookie
POST	`/api/auth/logout`	Clear the cookie

Candidate — authenticated by the access token in the path

Method	Path	Purpose
GET	`/api/interview/:token`	Room context and the join-gate verdict
GET	`/api/interview/:token/transcript`	Turns so far
GET	`/api/interview/:token/coding`	The coding exercise (hidden cases stripped)
POST	`/api/interview/:token/code/run`	Dry run or graded submission
POST	`/api/interview/:token/code/hint`	A single nudge, logged to the transcript
POST	`/api/interview/:token/video-metrics`	1–120 frames of aggregate numbers
GET / POST	`/api/interview/:token/resume`	Upload status / upload a resume

Recruiter — `requireAuth`

Method	Path	Purpose
GET	`/api/auth/me`	Current user
GET	`/api/sessions/config`	Providers, personalities, languages, experience levels
GET	`/api/sessions/upcoming`	Next eight interviews
POST	`/api/sessions/compare`	Compare 2–6 candidates
GET / POST	`/api/sessions`	List (search, filters, pagination) / create
GET / PATCH / DELETE	`/api/sessions/:id`	Read / update / delete
POST	`/api/sessions/:id/cancel`	Cancel and stop chasing candidates

Method	Path	Purpose
POST	`/api/sessions/:id/questions/generate`	Regenerate the set
POST / PATCH / DELETE	`/api/sessions/:id/questions[/:questionId]`	Add / edit / remove — refused once the interview has started
GET / POST	`/api/sessions/:id/candidates`	List / add
POST	`/api/sessions/:id/candidates/bulk`	CSV or XLSX upload
DELETE	`/api/sessions/:id/candidates/:candidateId`	Remove from the session
POST	`/api/sessions/:id/candidates/:candidateId/resend`	Re-queue the invite and tick immediately
GET	`/api/candidates`	Cross-session directory
GET	`/api/sessions/:id/leaderboard · /shortlist · /export.xlsx`	Ranking, shortlist, workbook
GET	`/api/interviews/queue`	Bot enablement, capacity, next launches, server time
GET	`/api/interviews/import/template`	CSV template (BOM-prefixed)
POST	`/api/interviews/import/preview`	Parse a file into rows with per-cell errors; writes nothing
GET / POST	`/api/interviews`	List meeting interviews / book one
POST	`/api/interviews/bulk`	Book up to 200, reporting each row's fate
GET	`/api/interviews/:id · /status · /transcript`	Full view · small polled view · numbered transcript
POST	`/api/interviews/:id/start · /stop · /report/retry`	Manual control
GET	`/api/interviews/:scId/insights · /evaluation · /report`	Live signals · evaluation state · report id
POST	`/api/interviews/:scId/evaluate`	Force a fresh evaluation, clearing the backoff
GET / POST	`/api/interviews/:scId/resume[/file]`	Parsed profile · original bytes · recruiter-side upload
GET	`/api/reports/recent · /:id`	Recent list · full report with grouped scores
GET	`/api/reports/:id/export.pdf · .xlsx`	Downloads
POST	`/api/reports/:id/feedback-email · /ats-sync`	Send candidate feedback · push to every enabled ATS
PATCH	`/api/reports/:id/recommendation`	Human override
GET	`/api/analytics/overview · /skills · /sessions`	Totals and trends · skill strengths · per-session table
GET / POST / DELETE	`/api/ats/integrations[/:id]`	Manage integrations
GET / POST	`/api/ats/integrations/:id/logs · /test`	Last 50 syncs · fire against the newest report

12.2 Socket.IO namespaces

Namespace	Auth	Direction and purpose
`/interview`	Access token in the handshake query, re-checked against the join gate	Bidirectional. The candidate's room: answers, doubts, audio chunks, listening gate, proctoring events; the AI's speech, state, insights, coding challenge, transcripts and terminal events.
`/meet-bot`	Recruiter JWT in handshake auth; ownership checked per subscription	**Read-only.** Status, joined, messages, questions, answers, interim text, completion and errors, with a 60-event replay buffer for a recruiter who opens the page late.
`/coding`	The same candidate access token	The editor broadcasts `code:update`, `code:running` and `code:submitted`; the spectator page renders them. Last state cached per interview so a late viewer sees code immediately.

12.3 Bot error codes

Thirty-two codes, each with recruiter-facing guidance. Retryable ones are marked `R`.

Code	Meaning
`INVALID_MEETING_URL` / `UNSUPPORTED_PLATFORM`	Not a link the interviewer recognises
`BOT_DISABLED` / `SERVERLESS_HOST`	Switched off, or a host where it fundamentally cannot work
`BROWSER_LAUNCH_FAILED` `R` / `BROWSER_CRASHED` `R`	Chromium could not start, or stopped mid-interview
`SIGN_IN_REQUIRED` / `VERIFICATION_REQUIRED` / `GUEST_JOIN_BLOCKED` / `PASSCODE_REQUIRED`	Identity problems a human must resolve; the bot never attempts them
`MEETING_PAGE_TIMEOUT` `R` / `PRE_JOIN_NOT_FOUND` `R` / `JOIN_CONTROL_NOT_FOUND` `R`	The page never got where it needed to be — usually a drifted selector, which is why a capture is saved
`ADMISSION_DENIED` / `ADMISSION_TIMEOUT`	The host declined, or nobody let the bot in
`MEETING_NOT_STARTED` `R` / `MEETING_ENDED` / `REMOVED_FROM_MEETING`	The meeting's own lifecycle
`CANDIDATE_NO_SHOW`	Nobody came; recorded as cancelled, not failed
`MICROPHONE_UNAVAILABLE` / `AUDIO_BRIDGE_FAILED` `R` / `TTS_UNAVAILABLE` / `SPEECH_TO_TEXT_UNAVAILABLE`	The interviewer cannot be heard, or cannot hear
`LLM_UNAVAILABLE` / `NETWORK_FAILURE` `R`	Upstream failures
`CAPACITY_REACHED` `R` / `ALREADY_RUNNING` / `NOT_STARTABLE` / `RUN_NOT_FOUND`	Scheduling states, surfaced as 409 rather than 400

Code	Meaning
`STOPPED_BY_RECRUITER / UNKNOWN`	Deliberate stop; anything unclassified

12.4 Interview rounds

```
GREETING → IDENTITY → INTRO → HR → TECHNICAL → SCENARIO → PROJECT → CODING → CLOSING
```

Which rounds appear depends on the interview type and whether coding is enabled. Within a round the AI may ask up to two follow-ups per question, and exactly one rephrase or clarification, before the machine forces progress.

12.5 A passing end-to-end run

```
28 AI turns · rounds: GREETING → IDENTITY → INTRO → HR → TECHNICAL →
                      SCENARIO → PROJECT → CLOSING
Overall 7.9 · Technical 7.6 · Communication 9.0 · Behavioral 7.0 ·
Video 7.9 → HIRE
55 transcript turns · 11 live signals · PDF/Excel exports · ranking ·
analytics
E2E PASSED
```

WHERE TO LOOK NEXT

`docs/MEETING_BOT.md` for the bot in operational detail, `docs/ENVIRONMENT.md` for every configuration value, and `docs/DEPLOYMENT*.md` for Render, Fly.io, Cloud Run and Koyeb walkthroughs. The `README.md` at the repository root is the condensed version of this document.